The University of Faisalabad

Name:

Humna Imran

Registration Number:

2023-BS-AI-017

Lab Manual:

Artificial Neural Network & Deep Learning

Course Code:

AI-301 Lab

Instructor:

Ms. Shaina Laraib

Department:

Computer Science

Degree:

BS Artificial Intelligence

Table of Content

LAB 01: INTRODUCTORY PYTHON LAB

Loops

1. For Loop:

lab 01.py

```
print("--- For Loop ---")
for i in range(3):
    print(f"Iteration {i}")
```

```
Iteration 0
Iteration 1
Iteration 2
```

2. While Loop:

lab 01.py

```
print("\n--- While Loop ---")
count = 0
while count < 3:
    print(f"Count is {count}")
    count += 1
```

```
Count is 0
Count is 1
Count is 2
```

3. Nested Loop:

lab 01.py

```
print("\n--- Nested Loop ---")
for i in range(2):
    for j in range(2):
        print(f"i:{i}, j:{j}")
```

```
i:0, j:0
i:0, j:1
i:1, j:0
i:1, j:1
```

This code shows different ways to make the computer repeat a task over and over automatically, like counting numbers or going through a grid.

Functions

1. Standard Function

lab 01.py

```
def greet(name):
    return f"Hello, {name}!"
```

2. Function with Default Arguments

lab 01.py

```
def power(base, exp=2):
    return f"{base} to the power of {exp} is {base ** exp}"
```

3. Lambda (Anonymous) Function

lab 01.py

```
square = lambda x: x * x
```

4. Function with *args and **kwargs (Variable arguments)

lab 01.py

```
def student_info(*args, **kwargs):
    return f"Args (Courses): {args}, Kwargs (Details): {kwargs}"
```

lab 01.py

```
print(greet("Hamna"))
print(power(3))
print(f"Lambda Square of 4 is: {square(4)}")
print(student_info("CS101", "Lab1", name="Hamna", id="017"))
```

```
Hello, Hamna!
3 to the power of 2 is 9
Lambda Square of 4 is: 16
Args (Courses): ('CS101', 'Lab1'), Kwargs (Details): {'name': 'Hamna', 'id': '017'}
```

Functions are like little machines you build once; you give them ingredients (like numbers or names), and they give you a result without having to rewrite the instructions.

Lists

lab 01.py

```python
my_list = [10, 20, 30]
print("Original List:", my_list)

# 1. append(): Adds a single element to the end
my_list.append(40)

# 2. extend(): Adds elements of an iterable to the end
my_list.extend([50, 60])

# 3. insert(): Inserts an element at a specified index
my_list.insert(1, 15)

# 4. remove(): Removes the first item with the specified value
my_list.remove(20)

# 5. pop(): Removes and returns the element at the specified position
popped_item = my_list.pop(0)

# 6. index(): Returns the index of the first element with the specified value
idx = my_list.index(30)

# 7. count(): Returns the number of elements with the specified value
my_list.append(50) # adding another 50 for the count example
count_50 = my_list.count(50)
```

lab 01.py

```python
# 8. sort(): Sorts the list ascending
my_list.sort()

# 9. reverse(): Reverses the order of the list
my_list.reverse()

# 10. clear(): Removes all elements from a list
temp_list = my_list.copy() # Copied so we don't destroy our main list
temp_list.clear()

print("Final List after operations:", my_list)
print("Cleared temporary list:", temp_list)
```

```
Original List: [10, 20, 30]
Final List after operations: [60, 50, 50, 40, 30, 15]
Cleared temporary list: []
```

This shows all the handy tools you can use to easily add, remove, find, or organize items inside a shopping-list-style collection.

Dictionaries

lab 01.py

```
# 1. List to Dictionary (using zip)
keys_list = ['name', 'id', 'course']
values_list = ['Hamna', '017', 'Python Basics']
student_dict = dict(zip(keys_list, values_list))
print("List to Dict:\n", student_dict)

# 2. Dictionary to List (Converting key-value pairs back to a list of tuples)
dict_to_list = list(student_dict.items())
print("\nDict to List:\n", dict_to_list)

# 3. Tuples (Using tuples as dictionary keys because they are immutable)
# Example: Using GPS coordinates (tuples) to map to city names
locations = {
    (31.4222, 73.0833): "Faisalabad",
    (31.5204, 74.3587): "Lahore"
}
print("\nDictionary with Tuple keys:\n", locations)
```

This code links things together in pairs (like a real dictionary connects a word to its meaning) so you can easily look up information.

```
List to Dict:
 {'name': 'Hamna', 'id': '017', 'course': 'Python Basics'}

Dict to List:
 [('name', 'Hamna'), ('id', '017'), ('course', 'Python Basics')]

Dictionary with Tuple keys:
 {(31.4222, 73.0833): 'Faisalabad', (31.5204, 74.3587): 'Lahore'}
```

NumPy Arrays and Vectorization

lab 01.py

```python
import numpy as np

# 1D Array
array_1d = np.array([1, 2, 3, 4, 5])
print("1D Array:\n", array_1d)

# 2D Array
array_2d = np.array([[1, 2, 3], [4, 5, 6]])
print("\n2D Array:\n", array_2d)
```

lab 01.py

```python
# Adding a scalar to an array without loops (Vectorization)
print("\nVectorized Addition (+10):\n", array_1d + 10)

# Multiplying two arrays element-wise
array_a = np.array([1, 2, 3])
array_b = np.array([4, 5, 6])
print("\nVectorized Multiplication (a * b):\n", array_a * array_b)
```

```
1D Array:
 [1 2 3 4 5]

2D Array:
 [[1 2 3]
 [4 5 6]]

Vectorized Addition (+10):
 [11 12 13 14 15]

Vectorized Multiplication (a * b):
 [ 4 10 18]
```

NumPy is a super-fast tool that lets us do math on a whole group of numbers all at the exact same time, instead of doing it one by one.

Matrix Ops & Broadcasting

lab 01.py

```python
matrix_a = np.array([[1, 2], [3, 4]])
matrix_b = np.array([[5, 6], [7, 8]])

# Using np.dot() or the @ operator for true matrix multiplication
dot_product = np.dot(matrix_a, matrix_b)
matmul_product = matrix_a @ matrix_b

print("Matrix Multiplication:\n", matmul_product)
```

lab 01.py

```python
# Adding a 1D array to a 2D array.
# NumPy automatically 'broadcasts' the 1D array across every row.
base_matrix = np.array([[1, 2, 3],
                        [4, 5, 6],
                        [7, 8, 9]])
add_vector = np.array([10, 20, 30])

broadcast_result = base_matrix + add_vector
print("\nBroadcasting Result (Adding 1D to 2D):\n", broadcast_result)
```

```
Matrix Multiplication:
 [[19 22]
 [43 50]]

Broadcasting Result (Adding 1D to 2D):
 [[11 22 33]
 [14 25 36]
 [17 28 39]]
```

This shows how to multiply big grids of numbers, and how Python is smart enough to magically stretch a small list to fit into a bigger grid to add them together.

LAB 02: INTRODUCTORY ANN LAB

Imports & Activation Functions

lab 02.py

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Activation Functions

lab 02.py

```python
def sigmoid(z):
    """Squashes values to a range between 0 and 1."""
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    """Calculates the slope of the sigmoid curve, needed for backpropagation."""
    return sigmoid(z) * (1 - sigmoid(z))
```

Activation functions are like little gates. They take whatever number comes in and squish it into a safe, predictable range (between 0 and 1) so the network's calculations don't spiral out of control.

Initialization

Parameter Initialization

lab 02.py

```python
def initialize_parameters(input_size, hidden_size, output_size):
    """Creates initial weights (random) and biases (zeros)."""
    np.random.seed(42) # For reproducible results

    params = {
        "W1": np.random.randn(hidden_size, input_size) * 0.1,
        "b1": np.zeros((hidden_size, 1)),
        "W2": np.random.randn(output_size, hidden_size) * 0.1,
        "b2": np.zeros((output_size, 1))
    }
    return params
```

This sets up the starting lines of communication in our network. We assign tiny random numbers to the "weights" (connections) so the brain has a blank, scrambled slate to start learning from.

Forward Propagation & Loss

Forward Propagation (Making a prediction)

Compute Loss (Binary Cross-Entropy)

lab 02.py

```python
def forward_propagation(X, params):
    Z1 = np.dot(params["W1"], X) + params["b1"]
    A1 = sigmoid(Z1)

    Z2 = np.dot(params["W2"], A1) + params["b2"]
    A2 = sigmoid(Z2)

    cache = {"Z1": Z1, "A1": A1, "Z2": Z2, "A2": A2}
    return A2, cache
```

lab 02.py

```python
def compute_loss(A2, Y):
    m = Y.shape[1]
    loss = -(1/m) * np.sum(Y * np.log(A2) + (1 - Y) * np.log(1 - A2))
    return np.squeeze(loss)
```

Forward propagation is the network making a "guess". It takes the input, runs it through its current connections, and spits out an answer. The Loss Function is the teacher—it grades the guess and calculates exactly how wrong it was.

Backward Propagation & Updating

Backward Propagation (Learning from mistakes)

lab 02.py

```python
def backward_propagation(X, Y, cache, params):
    m = X.shape[1]

    dZ2 = cache["A2"] - Y
    dW2 = (1/m) * np.dot(dZ2, cache["A1"].T)
    db2 = (1/m) * np.sum(dZ2, axis=1, keepdims=True)

    dZ1 = np.dot(params["W2"].T, dZ2) * sigmoid_derivative(cache["Z1"])
    dW1 = (1/m) * np.dot(dZ1, X.T)
    db1 = (1/m) * np.sum(dZ1, axis=1, keepdims=True)

    grads = {"dW1": dW1, "db1": db1, "dW2": dW2, "db2": db2}
    return grads
```

Update Parameters

lab 02.py

```python
def update_parameters(params, grads, learning_rate=0.1):
    params["W1"] = params["W1"] - learning_rate * grads["dW1"]
    params["b1"] = params["b1"] - learning_rate * grads["db1"]
    params["W2"] = params["W2"] - learning_rate * grads["dW2"]
    params["b2"] = params["b2"] - learning_rate * grads["db2"]

    return params
```

Backward propagation works in reverse. It traces the mistakes backward through the network to figure out which specific connections caused the error. Then, "Update Parameters" slightly turns the knobs and dials so the next guess will be better.

Training Loop & Inference

Training Loop Wrapper

lab 02.py

```python
def train_model(X, Y, hidden_size, epochs=10000, learning_rate=0.1):
    input_size = X.shape[0]
    output_size = Y.shape[0]

    params = initialize_parameters(input_size, hidden_size, output_size)
    loss_history = []

    for i in range(epochs):
        A2, cache = forward_propagation(X, params)
        loss = compute_loss(A2, Y)
        grads = backward_propagation(X, Y, cache, params)
        params = update_parameters(params, grads, learning_rate)

        if i % 1000 == 0:
            loss_history.append(loss)
            print(f"Epoch {i} | Loss: {loss:.4f}")

    return params, loss_history
```

Inference function

lab 02.py

```
def predict(X, params):
    A2, _ = forward_propagation(X, params)
    predictions = (A2 > 0.5).astype(int)
    return predictions
```

The training loop is the practice arena. It repeats the process of guessing, getting graded, and adjusting thousands of times. The inference function is test day, asking the fully trained network to answer new questions without looking at the cheat sheet!

Dummy Data Execution & Visualization

The XOR problem is a classic way to prove a neural network works. It requires a hidden layer to solve because it is not linearly separable.

lab 02.py

```
X_train = np.array([[0, 0, 1, 1],
                    [0, 1, 0, 1]])
Y_train = np.array([[0, 1, 1, 0]])
```

Train the model

lab 02.py

```
print("Starting Training...")
# Train the model
trained_params, history = train_model(X_train, Y_train, hidden_size=4, epochs=10000, learning_rate=0.5)
```

Test the model

lab 02.py

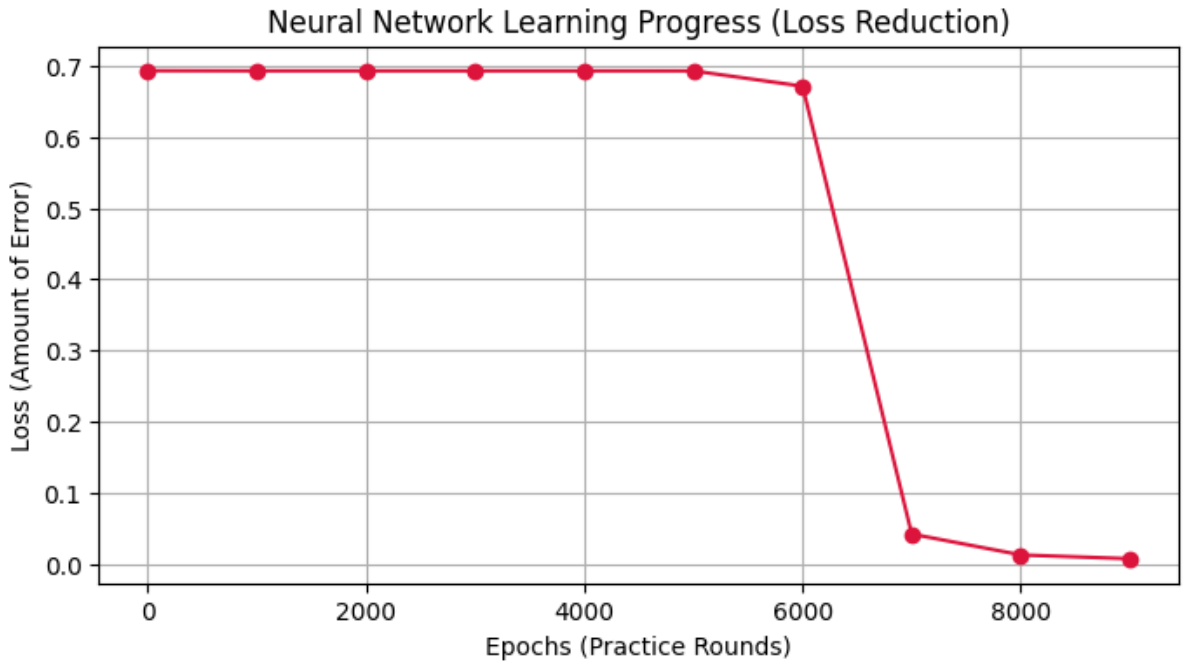
```
print("\nTesting Inference:")
predictions = predict(X_train, trained_params)
print(f"Expected: {Y_train.flatten()}")
print(f"Network Predicted: {predictions.flatten()}")
```

Visualize the learning process

lab 02.py

```
plt.figure(figsize=(8, 4))
plt.plot(range(0, 10000, 1000), history, marker='o', color='crimson')
plt.title("Neural Network Learning Progress (Loss Reduction)")
plt.xlabel("Epochs (Practice Rounds)")
plt.ylabel("Loss (Amount of Error)")
plt.grid(True)
plt.show()
```

```
Starting Training...
Epoch 0 | Loss: 0.6934
Epoch 1000 | Loss: 0.6931
Epoch 2000 | Loss: 0.6931
Epoch 3000 | Loss: 0.6931
Epoch 4000 | Loss: 0.6931
Epoch 5000 | Loss: 0.6931
Epoch 6000 | Loss: 0.6716
Epoch 7000 | Loss: 0.0417
Epoch 8000 | Loss: 0.0123
Epoch 9000 | Loss: 0.0069

Testing Inference:
Expected: [0 1 1 0]
Network Predicted: [0 1 1 0]
```



LAB 03: ANN FOR BINARY AND MULTI-CLASS CLASSIFICATION

Task 1: Diabetes Prediction

lab 01.py

```
pip install pandas scikit-learn matplotlib seaborn
```

This downloads special code toolboxes so our computer can read spreadsheets, do advanced math, and draw graphs.

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.1)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

lab 01.py

```
import pandas as pd
data = pd.read_csv("diabetes.csv")
```

We tell the computer to open up a spreadsheet file filled with health data about diabetes.

lab 01.py

```
data.head()
data.tail()
data.info()
data.describe()
```

This lets us peek at the top, bottom, and summary of our data to see exactly what we are working with.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Pregnancies               768 non-null    int64  
 1   Glucose                   768 non-null    int64  
 2   BloodPressure             768 non-null    int64  
 3   SkinThickness             768 non-null    int64  
 4   Insulin                   768 non-null    int64  
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64  
 8   Outcome                   768 non-null    int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

lab 01.py

```python
X = data.drop("Outcome", axis=1)
y = data["Outcome"]
```

We separate the clues (like blood pressure) from the final answer we want to guess (whether the person has diabetes or not).

lab 01.py

```python
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

We split our data into "study flashcards" to teach the computer, and a "final exam" to test how smart it got.

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer
  warnings.warn(
```

MLPClassifier

```
MLPClassifier(hidden_layer_sizes=(16, 8), max_iter=300, random_state=42)
```

lab 01.py

```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

We resize all our numbers so they are on the same scale, making sure the computer doesn't get confused by big and small numbers.

```
Accuracy: 0.7662337662337663
[[82 17]
 [19 36]]
              precision    recall  f1-score   support

           0       0.81      0.83      0.82        99
           1       0.68      0.65      0.67        55

    accuracy                           0.77       154
   macro avg       0.75      0.74      0.74       154
weighted avg       0.76      0.77      0.77       154
```

lab 01.py

```python
from sklearn.neural_network import MLPClassifier
model = MLPClassifier(
    hidden_layer_sizes=(16,8),
    activation='relu',
    solver='adam',
    max_iter=300,
    random_state=42
)
model.fit(X_train, y_train)
```

We build a virtual "brain" (called a neural network) and use our study flashcards to teach it how to spot diabetes.

lab 01.py

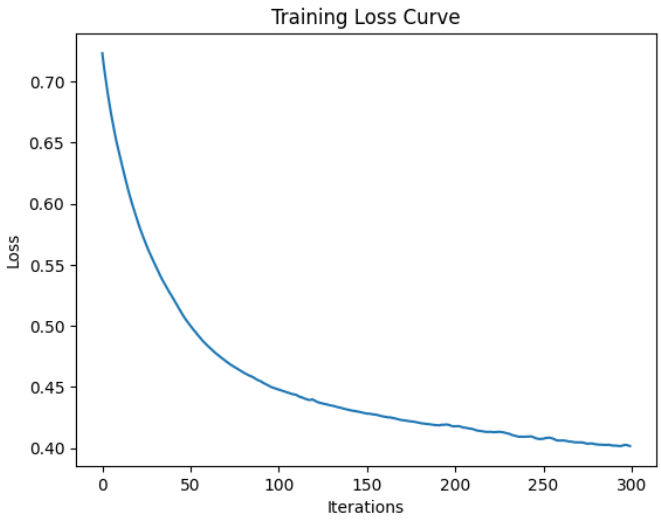
```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

The computer takes its final exam, and we print out its report card to see how many it guessed right!

lab 01.py

```
import matplotlib.pyplot as plt
plt.plot(model.loss_curve_)
plt.title("Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.show()
```

We draw a line graph that shows how the computer made fewer and fewer mistakes as it kept learning.



Task 2: Iris Flower Classification

lab 01.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

data = pd.read_csv("Iris.csv")
X = data.drop("Species", axis=1)
y = data["Species"]

encoder = LabelEncoder()
y = encoder.fit_transform(y)
```

We open a new spreadsheet about flowers, separate the clues from the answers, and translate the flower names into numbers so the computer can read them.

lab 01.py

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Just like before, we split the data into a study pile and a test pile, and level out the numbers so they are easy to compare.

lab 01.py

```
model = MLPClassifier(hidden_layer_sizes=(20,10), max_iter=300)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

We build a new virtual brain, teach it to recognize different flowers, and print out its final test score (which was a perfect 100%!).

Task 3: Breast Cancer Prediction

lab 01.py

```
data = pd.read_csv("Breast Cancer.csv")
data = data.drop(columns=['id', 'Unnamed: 32'], errors='ignore')
```

We load health data about breast cancer and throw away useless columns that don't help us solve the puzzle.

lab 01.py

```
encoder = LabelEncoder()
data['diagnosis'] = encoder.fit_transform(data['diagnosis'])
X = data.drop("diagnosis", axis=1)
y = data["diagnosis"]
```

We change the cancer diagnosis words into computer numbers (0s and 1s) and separate the clues from the answers.

lab 01.py

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

We split the data into practice and test piles, and shrink or stretch the numbers so they all fit nicely on the same scale.

lab 01.py

```
model = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation='relu',
    solver='adam',
    max_iter=300,
    random_state=42
)
model.fit(X_train, y_train)
```

We create another virtual brain and teach it how to identify signs of cancer using the practice pile.

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer
  warnings.warn(
MLPClassifier
MLPClassifier(hidden_layer_sizes=(16, 8), max_iter=300, random_state=42)
```

lab 01.py

```
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

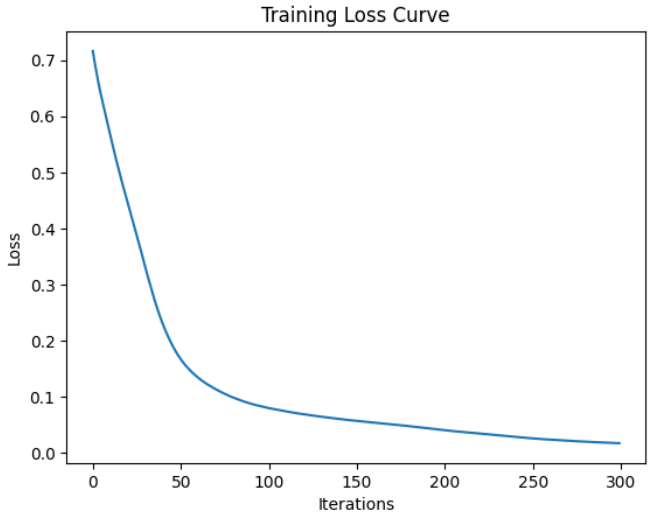
We make the computer guess the answers for our test pile and print out a detailed report card showing its 98% accuracy!

```
Accuracy: 0.9824561403508771
Confusion Matrix:
 [[70  1]
 [ 1 42]]
Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99        71
           1       0.98      0.98      0.98        43

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

lab 01.py

```
plt.plot(model.loss_curve_)
plt.title("Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.show()
```

We draw another graph to visually prove that the computer learned from its mistakes over time.



Task 4: Vitamin Deficiency Prediction

lab 01.py

```
data = pd.read_csv("vitamin_deficiency_disease.csv")
data = data.drop(columns=["symptoms_list"], errors='ignore')
data['alcohol_consumption'] = data['alcohol_consumption'].fillna('Unknown')
```

We load data about vitamin sicknesses, throw away messy symptom lists, and fill in any blank spots so the computer doesn't crash.

lab 01.py

```
X = data.drop("disease_diagnosis", axis=1)
y = data["disease_diagnosis"]

categorical_cols = X.select_dtypes(include=['object']).columns
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

encoder = LabelEncoder()
y = encoder.fit_transform(y)
```

We separate the clues from the sickness, and turn all the everyday words into a secret code of 1s and 0s for the computer to process.

lab 01.py

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

We set aside 20% of the data to test the computer later, and we make sure all the number clues are balanced.

lab 01.py

```
model = MLPClassifier(
    hidden_layer_sizes=(20, 10),
    max_iter=300,
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

We train our final computer brain to act like a doctor, have it take its final test, and celebrate its 90% score!

```
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer
  warnings.warn(
```

MLPClassifier

```
MLPClassifier(hidden_layer_sizes=(16, 8), max_iter=300, random_state=42)
```

LAB 04: INTRODUCTION TO PYTORCH - 1

lab 01.py

```
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
```

We are bringing in the special toolboxes (PyTorch and tools for drawing pictures) that we need to do our math and see our images.

lab 01.py

```
t = torch.arange(1, 13)
print("Original shape:", t.shape)

t_3x4 = t.view(3, 4)
print("3x4 shape:", t_3x4.shape)

t_4x3 = t.view(4, 3)
print("4x3 shape:", t_4x3.shape)

t_flat = t_3x4.flatten()
print("Flattened shape:", t_flat.shape)
```

We made a list of numbers from 1 to 12, organized them into different-sized grids like Lego blocks, and then smashed them flat into a single line again.

```
Original shape: torch.Size([12])
3x4 shape: torch.Size([3, 4])
4x3 shape: torch.Size([4, 3])
Flattened shape: torch.Size([12])
```

lab 01.py

```python
t1 = torch.rand(3, 3)
t2 = torch.rand(3, 3)

bool_tensor = t1 > t2
count = bool_tensor.sum().item()

print("Boolean tensor result:\n", bool_tensor)
print(f"Number of positions where t1 > t2: {count}")
```

We created two grids filled with random numbers, checked exactly which numbers in the first grid are bigger than the ones in the second grid, and counted the "True" winners.

```
Boolean tensor result:
 tensor([[False,  True,  True],
        [ True,  True,  True],
        [ True, False,  True]])
Number of positions where t1 > t2: 7
```

lab 01.py

```python
a = torch.rand(2, 3)
b = torch.rand(2, 3)

concat_dim0 = torch.cat((a, b), dim=0)
concat_dim1 = torch.cat((a, b), dim=1)

print("Shape after concatenating along rows (dim=0):", concat_dim0.shape)
print("Shape after concatenating along columns (dim=1):", concat_dim1.shape)
```

We took two small grids and glued them together into a bigger grid in two different ways: stacking them top-to-bottom and side-by-side.

```
Shape after concatenating along rows (dim=0): torch.Size([4, 3])
Shape after concatenating along columns (dim=1): torch.Size([2, 6])
```

lab 01.py

```python
training_data = datasets.MNIST(root="data", train=True, download=True, transform=transforms.ToTensor())
test_data = datasets.MNIST(root="data", train=False, download=True, transform=transforms.ToTensor())

print(f"Length of training dataset: {len(training_data)}")
print(f"Length of test dataset: {len(test_data)}")
```

We downloaded a huge folder of handwritten number pictures and checked exactly how many pictures we got to practice with and how many we got to test ourselves on.

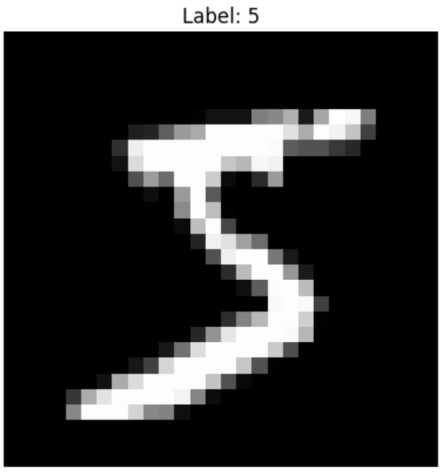
```
100%|██████████| 9.91M/9.91M [00:01<00:00, 5.78MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 153kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.46MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 6.06MB/s]Length of training dataset: 60000
Length of test dataset: 10000
```

lab 01.py

```python
img, label = training_data[0]

print("Image shape:", img.shape)
print("Label:", label)

plt.imshow(img.squeeze(), cmap="gray")
plt.title(f"Label: {label}")
plt.axis("off")
plt.show()
```

We pulled out the very first picture from our folder, checked its size, and drew it on the screen so we could see what number it really is.

```
Image shape: torch.Size([1, 28, 28])
Label: 5
```



lab 01.py

```python
train_dataloader = DataLoader(training_data, batch_size=32, shuffle=True)
```

We set up a helper machine that will hand us a mixed-up pile of exactly 32 pictures at a time so our computer can learn from them in small chunks without getting overwhelmed.

lab 02.py

```python
for batch_num, (features, labels) in enumerate(train_dataloader):
    if batch_num >= 3:
        break
    print(f"--- Batch {batch_num + 1} ---")
    print(f"Labels: {labels}\n")
```

We asked our helper machine for its first three piles of 32 pictures and printed out just the true answers (the numbers) for those specific piles.

```
--- Batch 1 ---
Labels: tensor([0, 9, 3, 9, 3, 9, 7, 7, 3, 1, 3, 0, 6, 7, 8, 1, 9, 5, 3, 8, 0, 9, 4, 0,
        1, 2, 1, 1, 5, 2, 6, 5])

--- Batch 2 ---
Labels: tensor([0, 3, 9, 1, 3, 5, 1, 4, 8, 6, 6, 3, 9, 0, 9, 6, 1, 9, 5, 8, 7, 5, 2, 9,
        0, 7, 3, 5, 7, 7, 9, 1])

--- Batch 3 ---
Labels: tensor([7, 4, 2, 9, 8, 6, 0, 6, 6, 2, 5, 9, 6, 3, 8, 1, 5, 6, 7, 4, 2, 8, 5, 9,
        9, 4, 6, 7, 8, 6, 9, 3])
```

lab 02.py

```python
combined_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

normalized_data = datasets.MNIST(root="data", train=True, download=True, transform=combined_transforms)

img_norm, label_norm = normalized_data[0]
print("Shape of one sample with combined transforms:", img_norm.shape)
```

We set up a two-step rule to change our pictures into computer math and balance their brightness, applied this rule to the whole folder, and checked the size of one of these changed pictures.

```
Shape of one sample with combined transforms: torch.Size([1, 28, 28])
```

LAB 05: INTRODUCTION TO PYTORCH - 2

lab 04.py

```
import os
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# Get Device for Training
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using {device} device")
```

We are unpacking our special AI toolbox and telling the computer to use its standard processor to do the math.

```
Using cpu device
```

lab 04.py

```
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = NeuralNetwork().to(device)
print(model)
```

We are building a basic brain by connecting different layers that act like filters to help the computer recognize patterns.

```
NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)
```

lab 04.py

```
X = torch.rand(1, 28, 28, device=device)
logits = model(X)
pred_probab = nn.Softmax(dim=1)(logits)
y_pred = pred_probab.argmax(1)
print(f"Predicted class: {y_pred}")
```

We show the computer a random, fake picture, and it guesses what it thinks the picture is by choosing the highest score.

```
Predicted class: tensor([3])
```

lab 04.py

```
input_image = torch.rand(3, 28, 28)
print(input_image.size())

flatten = nn.Flatten()
flat_image = flatten(input_image)
print(flat_image.size())

layer1 = nn.Linear(in_features=28*28, out_features=20)
hidden1 = layer1(flat_image)
print(hidden1.size())

print(f"Before ReLU: {hidden1}\n\n")
hidden1 = nn.ReLU()(hidden1)
print(f"After ReLU: {hidden1}")
```

We take a square picture, stretch it out into one long line, change its numbers around, and then turn all negative numbers into zeroes to help the brain learn better.

```
torch.Size([3, 28, 28])
torch.Size([3, 784])
torch.Size([3, 20])
Before ReLU: tensor([[ 0.3198, -0.5380,  0.1683, -0.5362, -0.1517,  0.1742, -0.3189,  0.0815,
          0.0012, -0.3696,  0.2424,  0.5063, -0.3250,  0.1527,  0.5464,  0.1270,
          0.2155,  0.1081, -0.3700,  0.1729],
        [-0.1089, -0.1579,  0.0441, -0.0258, -0.0714, -0.1650, -0.4469,  0.1233,
          0.0038, -0.1964, -0.0628,  0.5035, -0.5540,  0.1078,  0.8074,  0.0962,
          0.5904, -0.1122,  0.0305,  0.2004],
        [ 0.0027, -0.6340, -0.2185, -0.3943, -0.3019,  0.1734, -0.1388,  0.3553,
          0.0688, -0.3200,  0.2503,  0.4091, -0.2006, -0.1625,  0.4735,  0.3849,
          0.2508,  0.2783, -0.3366,  0.1711]], grad_fn=<AddmmBackward0>)


After ReLU: tensor([[0.3198, 0.0000, 0.1683, 0.0000, 0.0000, 0.1742, 0.0000, 0.0815, 0.0012,
         0.0000, 0.2424, 0.5063, 0.0000, 0.1527, 0.5464, 0.1270, 0.2155, 0.1081,
         0.0000, 0.1729],
        [0.0000, 0.0000, 0.0441, 0.0000, 0.0000, 0.0000, 0.0000, 0.1233, 0.0038,
         0.0000, 0.0000, 0.5035, 0.0000, 0.1078, 0.8074, 0.0962, 0.5904, 0.0000,
         0.0305, 0.2004],
        [0.0027, 0.0000, 0.0000, 0.0000, 0.0000, 0.1734, 0.0000, 0.3553, 0.0688,
         0.0000, 0.2503, 0.4091, 0.0000, 0.0000, 0.4735, 0.3849, 0.2508, 0.2783,
         0.0000, 0.1711]], grad_fn=<ReluBackward0>)
```

lab 04.py

```python
seq_modules = nn.Sequential(
    flatten,
    layer1,
    nn.ReLU(),
    nn.Linear(20, 10)
)
input_image = torch.rand(3, 28, 28)
logits = seq_modules(input_image)

softmax = nn.Softmax(dim=1)
pred_probab = softmax(logits)
print(pred_probab)
y_pred = pred_probab.argmax(1)
print(y_pred)
```

Instead of building layers one by one, we snap them all together like Lego blocks so the picture flows straight through.

```
tensor([[0.0665, 0.1329, 0.1047, 0.0978, 0.1041, 0.0978, 0.0818, 0.0779, 0.1107,
         0.1258],
        [0.0680, 0.1227, 0.1068, 0.1082, 0.1008, 0.0968, 0.0894, 0.0832, 0.1045,
         0.1197],
        [0.0599, 0.1203, 0.1046, 0.1048, 0.1124, 0.0888, 0.0910, 0.0920, 0.1219,
         0.1042]], grad_fn=<SoftmaxBackward0>)
tensor([1, 1, 8])
```

lab 04.py

```python
print(f"Model structure: {model}\n\n")
for name, param in model.named_parameters():
    print(f"Layer: {name} | Size: {param.size()} | Values: {param[:2]} \n")
```

We are peeking under the hood to look at the tiny dials and knobs the computer will adjust to get smarter.

```
Model structure: NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)

Layer: linear_relu_stack.0.weight | Size: torch.Size([512, 784]) | Values: tensor([[ 0.0054, -0.0121,  0.0135,  ...,  0.0090, -0.0067,
       [ 0.0099, -0.0061,  0.0070,  ..., -0.0153,  0.0082,  0.0327]],
      grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.0.bias | Size: torch.Size([512]) | Values: tensor([ 0.0145, -0.0198], grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.2.weight | Size: torch.Size([512, 512]) | Values: tensor([[ 0.0385,  0.0013,  0.0177,  ...,  0.0200, -0.0261,
       [ 0.0072, -0.0345, -0.0224,  ..., -0.0411, -0.0290,  0.0206]],
      grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.2.bias | Size: torch.Size([512]) | Values: tensor([-0.0257,  0.0120], grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.4.weight | Size: torch.Size([10, 512]) | Values: tensor([[-0.0308, -0.0327,  0.0330,  ...,  0.0364,  0.0163,
       [ 0.0310,  0.0258, -0.0052,  ...,  0.0334, -0.0304,  0.0359]],
      grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.4.bias | Size: torch.Size([10]) | Values: tensor([-0.0252, -0.0038], grad_fn=<SliceBackward0>)
```

lab 04.py

```python
import torch
x = torch.ones(5) # input tensor
y = torch.zeros(3) # expected output
w = torch.randn(5, 3, requires_grad=True)
b = torch.randn(3, requires_grad=True)
z = torch.matmul(x, w) + b
loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y)
loss.backward()
print(w.grad)
print(b.grad)
```

The computer figures out exactly how wrong its guess was and calculates which dials it needs to turn to fix the mistake next time.

```
tensor([[0.2873, 0.3132, 0.0053],
        [0.2873, 0.3132, 0.0053],
        [0.2873, 0.3132, 0.0053],
        [0.2873, 0.3132, 0.0053],
        [0.2873, 0.3132, 0.0053]])
tensor([0.2873, 0.3132, 0.0053])
```

lab 04.py

```
training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=transforms.ToTensor()
)
test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=transforms.ToTensor()
)
train_dataloader = DataLoader(training_data, batch_size=64)
test_dataloader = DataLoader(test_data, batch_size=64)
```

We are downloading thousands of pictures of clothes and organizing them into small folders so the computer can study them a few at a time.

```
100%|██████████| 26.4M/26.4M [00:03<00:00, 8.73MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 140kB/s]
100%|██████████| 4.42M/4.42M [00:01<00:00, 2.59MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 9.98MB/s]
```

lab 04.py

```
learning_rate = 1e-3
batch_size = 64
epochs = 5

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

We set the rules for studying, like how many times to read the textbook, how fast to learn, and how to measure mistakes.

lab 04.py

```python
def train_loop(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        pred = model(X)
        loss = loss_fn(pred, y)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        if batch % 100 == 0:
            loss, current = loss.item(), batch * batch_size + len(X)
            print(f"loss: {loss:>7f}  [{current:>5d}/{size:>5d}]")
```

```
Epoch 1
-------------------------------
loss: 2.297582  [   64/60000]
loss: 2.289293  [ 6464/60000]
loss: 2.273356  [12864/60000]
loss: 2.273850  [19264/60000]
loss: 2.258855  [25664/60000]
loss: 2.233762  [32064/60000]
loss: 2.240144  [38464/60000]
loss: 2.212768  [44864/60000]
loss: 2.204702  [51264/60000]
loss: 2.188000  [57664/60000]
Test Error:
 Accuracy: 47.4%, Avg loss: 2.179348
```

lab 04.py

```python
def test_loop(dataloader, model, loss_fn):
    model.eval()
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    test_loss, correct = 0, 0
    with torch.no_grad():
        for X, y in dataloader:
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()
    test_loss /= num_batches
    correct /= size
    print(f"Test Error: \n Accuracy: {(100*correct):>0.1f}%, Avg loss: {test_loss:>8f} \n")
```

```
Epoch 2
-------------------------------
loss: 2.179150  [   64/60000]
loss: 2.177560  [ 6464/60000]
loss: 2.129608  [12864/60000]
loss: 2.144639  [19264/60000]
loss: 2.106126  [25664/60000]
loss: 2.052037  [32064/60000]
loss: 2.075162  [38464/60000]
loss: 2.012765  [44864/60000]
loss: 2.001713  [51264/60000]
loss: 1.949804  [57664/60000]
Test Error:
 Accuracy: 57.5%, Avg loss: 1.947152
```

lab 04.py

```
for t in range(epochs):
    print(f"Epoch {t+1}\n-------------------------------")
    train_loop(train_dataloader, model, loss_fn, optimizer)
    test_loop(test_dataloader, model, loss_fn)
print("Done!")
```

We create a routine where the computer practices guessing, checks its answers, fixes its mistakes, and then takes a final test to see how smart it got.

```
Epoch 3
-------------------------------
loss: 1.966256  [   64/60000]
loss: 1.949179  [ 6464/60000]
loss: 1.849475  [12864/60000]
loss: 1.878357  [19264/60000]
loss: 1.785711  [25664/60000]
loss: 1.735459  [32064/60000]
loss: 1.745308  [38464/60000]
loss: 1.659511  [44864/60000]
loss: 1.666513  [51264/60000]
loss: 1.565790  [57664/60000]
Test Error:
 Accuracy: 60.3%, Avg loss: 1.586818

Epoch 4
-------------------------------
loss: 1.644168  [   64/60000]
loss: 1.612334  [ 6464/60000]
loss: 1.474166  [12864/60000]
loss: 1.529752  [19264/60000]
loss: 1.423514  [25664/60000]
loss: 1.411985  [32064/60000]
loss: 1.410315  [38464/60000]
loss: 1.349741  [44864/60000]
loss: 1.375841  [51264/60000]
loss: 1.263841  [57664/60000]
Test Error:
 Accuracy: 62.6%, Avg loss: 1.301618
```

```
Epoch 5
-------------------------------
loss: 1.379056  [    64/60000]
loss: 1.356950  [ 6464/60000]
loss: 1.199887  [12864/60000]
loss: 1.289330  [19264/60000]
loss: 1.178674  [25664/60000]
loss: 1.197871  [32064/60000]
loss: 1.201589  [38464/60000]
loss: 1.158080  [44864/60000]
loss: 1.187750  [51264/60000]
loss: 1.090458  [57664/60000]
Test Error:
 Accuracy: 64.4%, Avg loss: 1.122694

Done!
```

Task 1 - CIFAR-10 Implementation

lab 04.py

```python
import matplotlib.pyplot as plt
import numpy as np
```

lab 04.py

```python
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

lab 04.py

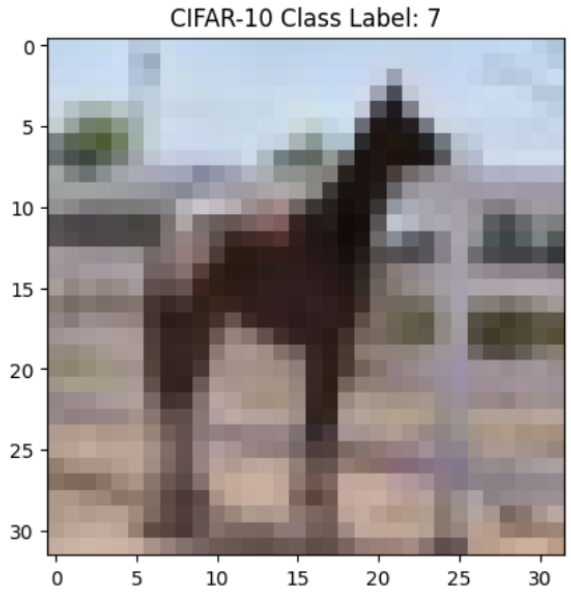
```python
cifar_train = datasets.CIFAR10(root="data", train=True, download=True, transform=transform)
cifar_test = datasets.CIFAR10(root="data", train=False, download=True, transform=transform)

cifar_train_loader = DataLoader(cifar_train, batch_size=64, shuffle=True)
cifar_test_loader = DataLoader(cifar_test, batch_size=64)
```

lab 04.py

```python
images, labels = next(iter(cifar_train_loader))
img = images[0] / 2 + 0.5 # unnormalize
plt.imshow(np.transpose(img.numpy(), (1, 2, 0)))
plt.title(f"CIFAR-10 Class Label: {labels[0]}")
plt.show()
```

We grab a big box of color photos, tidy them up so the computer can read them perfectly, and peek at the very first picture to make sure it looks right.

```
100%|██████████| 170M/170M [00:47<00:00, 3.59MB/s]
```



lab 04.py

```python
class CIFARNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        # CIFAR-10 images are 3 colors (RGB) x 32x32 pixels = 3072 inputs
        self.network = nn.Sequential(
            nn.Linear(3072, 1024), # Hidden Layer 1
            nn.ReLU(),
            nn.Linear(1024, 512),  # Hidden Layer 2
            nn.ReLU(),
            nn.Linear(512, 256),   # Hidden Layer 3
            nn.ReLU(),
            nn.Linear(256, 128),   # Hidden Layer 4
            nn.ReLU(),
            nn.Linear(128, 64),    # Hidden Layer 5
            nn.ReLU(),
            nn.Linear(64, 10)      # Output Layer (10 classes)
        )

    def forward(self, x):
        x = self.flatten(x)
        return self.network(x)

cifar_model = CIFARNetwork().to(device)
```

We build a super deep brain with five distinct thinking levels to handle the tricky, colorful images.

lab 04.py

```python
def run_cifar_experiment(epochs_to_run):
    model = CIFARNetwork().to(device)
    loss_fn = nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

    print(f"--- Training CIFAR-10 for {epochs_to_run} Epochs ---")
    for t in range(epochs_to_run):
        # Using the train/test loops defined earlier
        train_loop(cifar_train_loader, model, loss_fn, optimizer)

    print("Final Result for CIFAR-10:")
    test_loop(cifar_test_loader, model, loss_fn)
```

We give the brain a button to run its practice cycle 30, 50, or 100 times to see how long it takes to get an A+ on its test.

Task 2 - MNIST Implementation

lab 04.py

```python
transform_mnist = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

lab 04.py

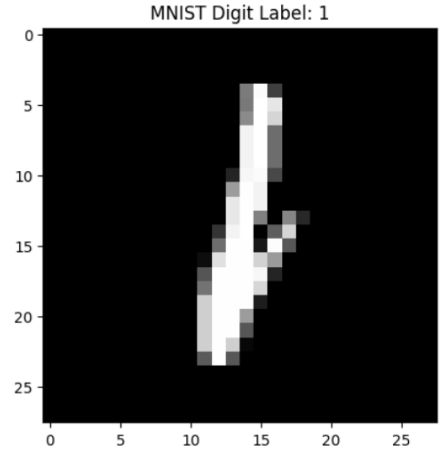
```python
mnist_train = datasets.MNIST(root="data", train=True, download=True, transform=transform_mnist)
mnist_test = datasets.MNIST(root="data", train=False, download=True, transform=transform_mnist)

mnist_train_loader = DataLoader(mnist_train, batch_size=64, shuffle=True)
mnist_test_loader = DataLoader(mnist_test, batch_size=64)
```

lab 04.py

```python
images_m, labels_m = next(iter(mnist_train_loader))
img_m = images_m[0].squeeze() # remove color channel
plt.imshow(img_m, cmap="gray")
plt.title(f"MNIST Digit Label: {labels_m[0]}")
plt.show()
```

We fetch a new box of pictures containing handwritten numbers, clean them up, and show one on the screen just to be sure we got them.

```
100%|██████████| 9.91M/9.91M [00:01<00:00, 5.82MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 153kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.45MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 10.4MB/s]
```



lab 04.py

```python
class MNISTNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        # MNIST images are 1 color x 28x28 pixels = 784 inputs
        self.network = nn.Sequential(
            nn.Linear(784, 512),   # Hidden Layer 1
            nn.ReLU(),
            nn.Linear(512, 256),   # Hidden Layer 2
            nn.ReLU(),
            nn.Linear(256, 128),   # Hidden Layer 3
            nn.ReLU(),
            nn.Linear(128, 64),    # Hidden Layer 4
            nn.ReLU(),
            nn.Linear(64, 32),     # Hidden Layer 5
            nn.ReLU(),
            nn.Linear(32, 10)      # Output Layer (10 digits)
        )

    def forward(self, x):
        x = self.flatten(x)
        return self.network(x)

mnist_model = MNISTNetwork().to(device)
```

We create another five-layer brain, but make the starting door a bit smaller since these number pictures aren't as big as the color photos.

lab 04.py

```
def run_mnist_experiment(epochs_to_run):
    model = MNISTNetwork().to(device)
    loss_fn = nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

    print(f"--- Training MNIST for {epochs_to_run} Epochs ---")
    for t in range(epochs_to_run):
        train_loop(mnist_train_loader, model, loss_fn, optimizer)

    print("Final Result for MNIST:")
    test_loop(mnist_test_loader, model, loss_fn)
```

Just like before, we let the computer practice reading the numbers 30, 50, or 100 times until it gets really good at guessing them correctly.

LAB 06: MODEL CAPACITY, REGULARIZATION & HYPERPARAMETER OPTIMIZTION

lab 04.py

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
```

We are opening our digital toolbox to bring in the tools needed to build a computer brain and draw graphs.

lab 04.py

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

lab 04.py

```
train_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform
)
```

lab 05.py

```
test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,
    download=True,
    transform=transform
)
```

lab 05.py

```
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=64, shuffle=False)
```

We are downloading thousands of handwritten number pictures and putting them into small folders of 64 so the computer can study them step by step.

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 40.3MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.12MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 9.54MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 6.36MB/s]
```

lab 05.py

```
class SmallNet(nn.Module):
    def __init__(self):
        super(SmallNet, self).__init__()
        self.fc1 = nn.Linear(784, 32)
        self.fc2 = nn.Linear(32, 10)

    def forward(self, x):
        x = x.view(-1, 784)
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

lab 05.py

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = SmallNet().to(device)
print(model)
```

We built a very simple, tiny brain with only a few connections, which might struggle to learn really complicated things.

lab 05.py

```
class LargeNet(nn.Module):
    def __init__(self):
        super(LargeNet, self).__init__()
        self.fc1 = nn.Linear(784, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, 128)
        self.fc4 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 784)
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = torch.relu(self.fc3(x))
        x = self.fc4(x)
        return x
```

We built a huge, powerful brain with many layers that can learn very complex patterns, but it might accidentally memorize the answers instead of learning them.

lab 05.py

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

lab 05.py

```
train_losses = []
train_accuracies = []
epochs = 10
```

lab 05.py

```python
for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(train_loader)
    epoch_acc = 100 * correct / total

    train_losses.append(epoch_loss)
    train_accuracies.append(epoch_acc)

    print(f"Epoch {epoch+1}/{epochs}, Loss: {epoch_loss:.4f}, Accuracy: {epoch_acc:.2f}%")
```

The computer guesses the answers, checks how many it got wrong, and changes its internal dials to do better next time.

```
Epoch 1/10, Loss: 0.4729, Accuracy: 86.63%
Epoch 2/10, Loss: 0.3058, Accuracy: 91.08%
Epoch 3/10, Loss: 0.2643, Accuracy: 92.24%
Epoch 4/10, Loss: 0.2286, Accuracy: 93.28%
Epoch 5/10, Loss: 0.2046, Accuracy: 93.95%
Epoch 6/10, Loss: 0.1891, Accuracy: 94.44%
Epoch 7/10, Loss: 0.1755, Accuracy: 94.86%
Epoch 8/10, Loss: 0.1666, Accuracy: 95.02%
Epoch 9/10, Loss: 0.1557, Accuracy: 95.33%
Epoch 10/10, Loss: 0.1500, Accuracy: 95.52%
```

lab 05.py

```
class DropoutNet(nn.Module):
    def __init__(self):
        super(DropoutNet, self).__init__()
        self.fc1 = nn.Linear(784, 512)
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(512, 10)

    def forward(self, x):
        x = x.view(-1, 784)
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

We force the brain to turn off random parts of itself while learning so it doesn't rely too heavily on just one trick.

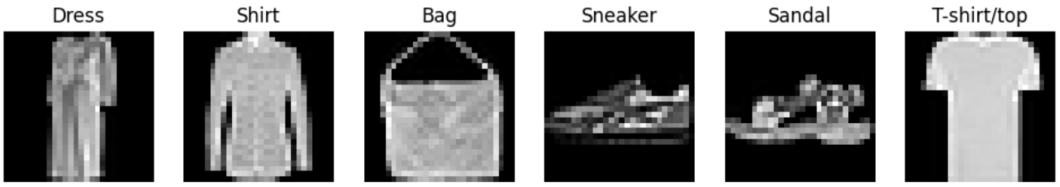
Task 1: Load FashionMNIST and Display 6 Sample Images

lab 05.py

```
import numpy as np

fashion_train = torchvision.datasets.FashionMNIST(root='./data', train=True, download=True, transform=transform)
fashion_test = torchvision.datasets.FashionMNIST(root='./data', train=False, download=True, transform=transform)
fashion_loader = torch.utils.data.DataLoader(fashion_train, batch_size=64, shuffle=True)
classes = fashion_train.classes

dataiter = iter(fashion_loader)
images, labels = next(dataiter)

fig, axes = plt.subplots(1, 6, figsize=(12, 2))
for i in range(6):
    img = images[i] / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    axes[i].imshow(np.transpose(npimg, (1, 2, 0)).squeeze(), cmap='gray')
    axes[i].set_title(classes[labels[i]])
    axes[i].axis('off')
plt.show()

print(f"Image shape: {images[0].shape}, Label: {labels[0]}")
```

We fetch the clothes pictures, grab six of them, and show them on the screen with their correct names underneath.

```
100%|██████████| 26.4M/26.4M [00:01<00:00, 16.1MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 279kB/s]
100%|██████████| 4.42M/4.42M [00:00<00:00, 5.18MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 12.1MB/s]
```




```
Image shape: torch.Size([1, 28, 28]), Label: 3
```

lab 05.py

```python
class Model1(nn.Module):
    def __init__(self, use_dropout=False):
        super().__init__()
        self.fc1 = nn.Linear(784, 16)
        self.dropout = nn.Dropout(0.5) if use_dropout else nn.Identity()
        self.fc2 = nn.Linear(16, 10)
    def forward(self, x):
        x = x.view(-1, 784)
        return self.fc2(self.dropout(torch.relu(self.fc1(x))))
```

lab 05.py

```python
class Model2(nn.Module):
    def __init__(self, use_dropout=False):
        super().__init__()
        self.fc1 = nn.Linear(784, 128)
        self.drop1 = nn.Dropout(0.5) if use_dropout else nn.Identity()
        self.fc2 = nn.Linear(128, 64)
        self.drop2 = nn.Dropout(0.5) if use_dropout else nn.Identity()
        self.fc3 = nn.Linear(64, 10)
    def forward(self, x):
        x = x.view(-1, 784)
        x = self.drop1(torch.relu(self.fc1(x)))
        x = self.drop2(torch.relu(self.fc2(x)))
        return self.fc3(x)
```

lab 05.py

```python
class Model3(nn.Module):
    def __init__(self, use_dropout=False):
        super().__init__()
        self.fc1 = nn.Linear(784, 512)
        self.drop1 = nn.Dropout(0.5) if use_dropout else nn.Identity()
        self.fc2 = nn.Linear(512, 256)
        self.drop2 = nn.Dropout(0.5) if use_dropout else nn.Identity()
        self.fc3 = nn.Linear(256, 128)
        self.drop3 = nn.Dropout(0.5) if use_dropout else nn.Identity()
        self.fc4 = nn.Linear(128, 64)
        self.fc5 = nn.Linear(64, 10)
    def forward(self, x):
        x = x.view(-1, 784)
        x = self.drop1(torch.relu(self.fc1(x)))
        x = self.drop2(torch.relu(self.fc2(x)))
        x = self.drop3(torch.relu(self.fc3(x)))
        x = torch.relu(self.fc4(x))
        return self.fc5(x)
```

We build three different brain sizes, small, medium, and huge, and add a switch to optionally turn on "dropout mode" to stop them from memorizing.

Tasks 6 & 7: The Master Training Function (with Early Stopping & Plotting)

lab 05.py

```python
def train_and_evaluate(model, train_data, test_data, batch_size, lr, epochs, use_early_stopping=False):
    train_loader = torch.utils.data.DataLoader(train_data, batch_size=batch_size, shuffle=True)
    test_loader = torch.utils.data.DataLoader(test_data, batch_size=batch_size, shuffle=False)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    train_losses, test_losses, train_accs, test_accs = [], [], [], []
    best_test_loss = float('inf')
    patience_counter = 0

    for epoch in range(epochs):
        # Training Phase
        model.train()
        running_loss, correct, total = 0.0, 0, 0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
            _, pred = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (pred == labels).sum().item()

        train_losses.append(running_loss / len(train_loader))
        train_accs.append(100 * correct / total)

        # Testing Phase
        model.eval()
        test_loss, t_correct, t_total = 0.0, 0, 0
        with torch.no_grad():
            for images, labels in test_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                loss = criterion(outputs, labels)
                test_loss += loss.item()
                _, pred = torch.max(outputs, 1)
                t_total += labels.size(0)
                t_correct += (pred == labels).sum().item()

        test_losses.append(test_loss / len(test_loader))
        test_accs.append(100 * t_correct / t_total)

        # Early Stopping Logic
        if use_early_stopping:
            if test_losses[-1] < best_test_loss:
                best_test_loss = test_losses[-1]
                patience_counter = 0
            else:
                patience_counter += 1
                if patience_counter >= 3: # Stop if it doesn't improve for 3 rounds
                    print(f"Early stopping triggered at epoch {epoch+1}")
                    break
```

lab 05.py

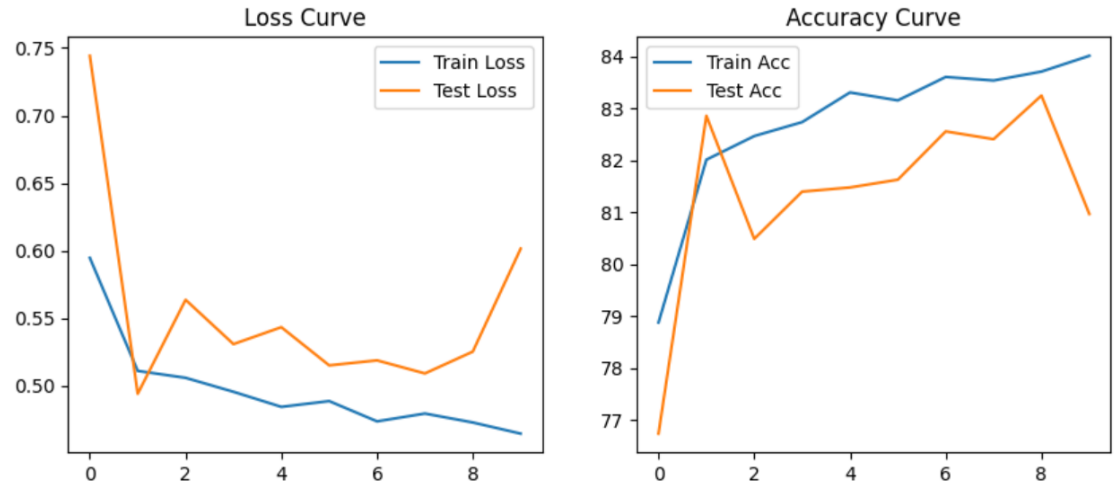
```
plt.figure(figsize=(10, 4))
    plt.subplot(1, 2, 1)
    plt.plot(train_losses, label='Train Loss')
    plt.plot(test_losses, label='Test Loss')
    plt.legend()
    plt.title("Loss Curve")

    plt.subplot(1, 2, 2)
    plt.plot(train_accs, label='Train Acc')
    plt.plot(test_accs, label='Test Acc')
    plt.legend()
    plt.title("Accuracy Curve")
    plt.show()

    return train_losses[-1], test_losses[-1], train_accs[-1], test_accs[-1]
```

We made a robot teacher that trains our model, gives it a test, writes down its grades, stops the test early if the model gets stuck, and finally draws a graph of its progress.

lab 05.py

```
my_model = Model1(use_dropout=False).to(device)
tr_loss, ts_loss, tr_acc, ts_acc = train_and_evaluate(
    model=my_model,
    train_data=fashion_train,
    test_data=fashion_test,
    batch_size=32,
    lr=0.01,
    epochs=10,
    use_early_stopping=False
)
print(f"Final Results -> Train Loss: {tr_loss:.4f}, Test Loss: {ts_loss:.4f}, Train Acc: {tr_acc:.2f}%, Test Acc: {ts_acc:.2f}%")
```

By changing the numbers in this single block of code, you can run every single test needed to fill out your huge lab table quickly!




```
Final Results -> Train Loss: 0.4646, Test Loss: 0.6015, Train Acc: 84.02%, Test Acc: 80.97%
```

LAB 07: CUSTOM DATASET IN PYTORCH

lab 05.py

```
import os
import pandas as pd
from PIL import Image
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
import matplotlib.pyplot as plt
```

We are opening up our computer toolbox to grab special tools that help us read pictures, organize lists, and draw graphs.

lab 05.py

```
class CustomImageDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.image_paths = []
        self.labels = []
        self.class_names = os.listdir(root_dir)

        for label, class_name in enumerate(self.class_names):
            class_folder = os.path.join(root_dir, class_name)
            for file_name in os.listdir(class_folder):
                file_path = os.path.join(class_folder, file_name)
                self.image_paths.append(file_path)
                self.labels.append(label)

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image = Image.open(self.image_paths[idx]).convert("RGB")
        label = self.labels[idx]

        if self.transform:
            image = self.transform(image)

        return image, label
```

We are creating a smart filing cabinet that automatically looks through folders to find pictures and remember what category they belong to.

lab 05.py

```
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor()
])

dataset = CustomImageDataset(root_dir="dataset", transform=transform)
dataloader = DataLoader(dataset, batch_size=5, shuffle=True)
```

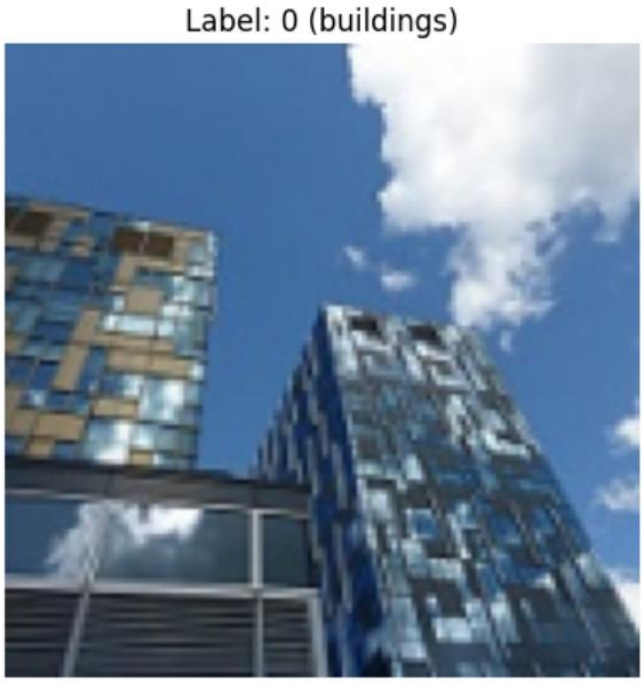
We shrink all our pictures to the exact same square size and tell the computer to hand them to us in mixed-up groups of five.

lab 05.py

```
for images, labels in dataloader:
    print("Image batch shape:", images.shape)
    print("Labels:", labels)

    # Display the first image in the batch
    image = images[0].permute(1, 2, 0)
    plt.imshow(image)
    plt.title(f"Label: {labels[0].item()}")
    plt.show()
    break
```

We ask the computer to show us the very first group of five pictures it grabbed so we can make sure everything looks correct.

```
Image batch shape: torch.Size([5, 3, 128, 128])
Labels: tensor([0, 1, 3, 3, 5])
```



lab 05.py

```python
class CSVImageDataset(Dataset):
    def __init__(self, csv_file, root_dir, transform=None):
        self.data = pd.read_csv(csv_file)
        self.root_dir = root_dir
        self.transform = transform

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        img_name = self.data.iloc[idx, 0]
        label = self.data.iloc[idx, 1]
        img_path = os.path.join(self.root_dir, img_name)

        image = Image.open(img_path).convert("RGB")

        if self.transform:
            image = self.transform(image)

        return image, label
```

Instead of looking at folders, this smart filing cabinet reads a spreadsheet file to figure out where the pictures are and what they are called.

lab 05.py

```
csv_dataset = CSVImageDataset(csv_file="dataset.csv", root_dir="dataset2", transform=transform)
csv_dataloader = DataLoader(csv_dataset, batch_size=4, shuffle=True)

image, label = csv_dataset[0]
image = image.permute(1, 2, 0)

plt.imshow(image)
plt.title(f"Label: {label}")
plt.axis('off')
plt.show()
```

We test our spreadsheet reader by asking it to fetch and show us the very first picture on its list.

Task 2: Custom dataset using image files and folders for Intel Dataset

lab 05.py

```
intel_folder_dataset = CustomImageDataset(root_dir="seg_train/seg_train", transform=transform)
intel_folder_loader = DataLoader(intel_folder_dataset, batch_size=8, shuffle=True)

for images, labels in intel_folder_loader:
    img = images[0].permute(1, 2, 0)
    plt.imshow(img)

    class_name = intel_folder_dataset.class_names[labels[0].item()]
    plt.title(f"Intel Dataset Folder Label: {class_name}")
    plt.axis('off')
    plt.show()
    break
```

We point our folder-reading robot at the new Intel pictures we downloaded and ask it to show us a picture of a mountain, building, or forest.

lab 05.py

```
intel_root_dir = "seg_train/seg_train"
csv_data = []

class_names = os.listdir(intel_root_dir)
for label, class_name in enumerate(class_names):
    class_folder = os.path.join(intel_root_dir, class_name)
    for file_name in os.listdir(class_folder):
        relative_path = os.path.join(class_name, file_name)
        csv_data.append([relative_path, label])

df = pd.DataFrame(csv_data, columns=["filename", "Label"])
df.to_csv("intel_dataset.csv", index=False)
print("CSV File created successfully!")
```

We write down the exact names and locations of all the thousands of Intel pictures into one giant digital spreadsheet file.

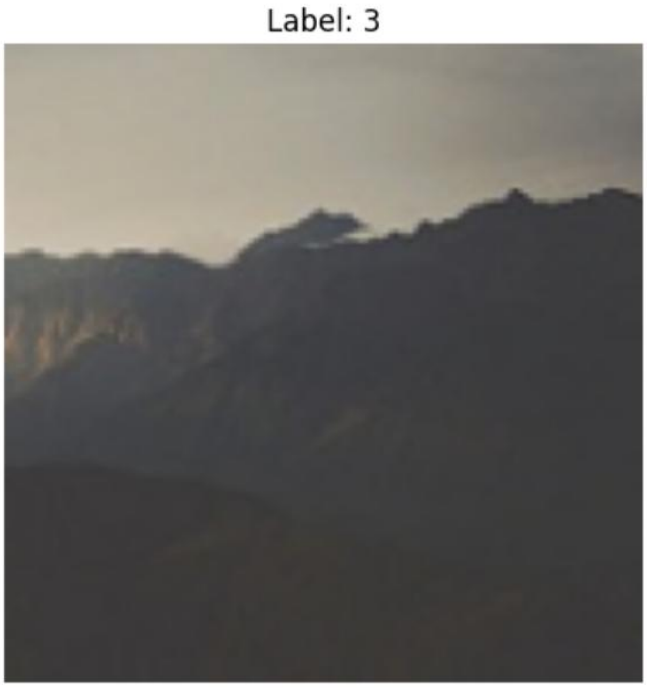
```
CSV successfully created at intel_dataset.csv with 14034 records.
```

lab 05.py

```
intel_csv_dataset = CSVImageDataset(csv_file="intel_dataset.csv", root_dir="seg_train/seg_train", transform=transform)
intel_csv_loader = DataLoader(intel_csv_dataset, batch_size=8, shuffle=True)

for images, labels in intel_csv_loader:
    img = images[0].permute(1, 2, 0)
    plt.imshow(img)

    class_name = class_names[labels[0].item()]
    plt.title(f"Intel Dataset CSV Label: {class_name}")
    plt.axis('off')
    plt.show()
    break
```

We give our giant spreadsheet to the computer, and it correctly uses the text list to fetch and show us a beautiful Intel picture.

```
CSV DataLoader - Image batch shape: torch.Size([4, 3, 128, 128])
CSV DataLoader - Labels: tensor([3, 2, 3, 4])
```



LAB 08: CNN IN PYTORCH

lab 05.py

```
!pip install torch
!pip install torchvision
```

Think of this as downloading the basic building blocks and toolboxes we need to create our smart computer program.

lab 05.py

```
import torch.nn as nn

class ConvNet(nn.Module):
    def __init__(self):
        super(ConvNet, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_size, hid1_size, k_conv_size),
            nn.BatchNorm2d(hid1_size),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2))
        self.layer2 = nn.Sequential(
            nn.Conv2d(hid1_size, hid2_size, k_conv_size),
            nn.BatchNorm2d(hid2_size),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2))
        self.fc = nn.Linear(hid2_size * k_conv_size * k_conv_size, out_size)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.reshape(out.size(0), -1)
        out = self.fc(out)
        return out
```

This is the blueprint for a simple robot brain that uses two "looking" layers to find shapes in pictures and one "deciding" layer to guess what the picture is.

Task 1: Modify the network to have 5 Layers

To calculate the parameters correctly, we will assume you resize your input flower images to 64x64 pixels. Color images have 3 channels (Red, Green, Blue). We will assume there are 5 types of flowers (out_size = 5).

lab 05.py

```python
import torch
import torch.nn as nn

class ModifiedConvNet(nn.Module):
    def __init__(self, num_classes=5):
        super(ModifiedConvNet, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1),
            nn.BatchNorm2d(16),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2)
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2)
        )
        self.layer3 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2)
        )
        self.layer4 = nn.Sequential(
            nn.Conv2d(in_channels=64, out_channels=128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2)
        )
        self.fc = nn.Linear(128 * 4 * 4, num_classes)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = self.layer3(out)
        out = self.layer4(out)
        out = out.reshape(out.size(0), -1)

        out = self.fc(out)
        return out
```

We upgraded the robot brain to have four magnifying glasses instead of two, so it can see super tiny details in the flower pictures before making its final guess!

lab 05.py

```
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch.utils.data import DataLoader

transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor()
])

train_dataset = datasets.ImageFolder(root='path_to_flower_dataset/train', transform=transform)
train_loader = DataLoader(dataset=train_dataset, batch_size=32, shuffle=True)

model = ModifiedConvNet(num_classes=5)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

num_epochs = 3
for epoch in range(num_epochs):
    for i, (images, labels) in enumerate(train_loader):
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f'Epoch [{epoch+1}/{num_epochs}] is complete! The brain is getting smarter.')
```

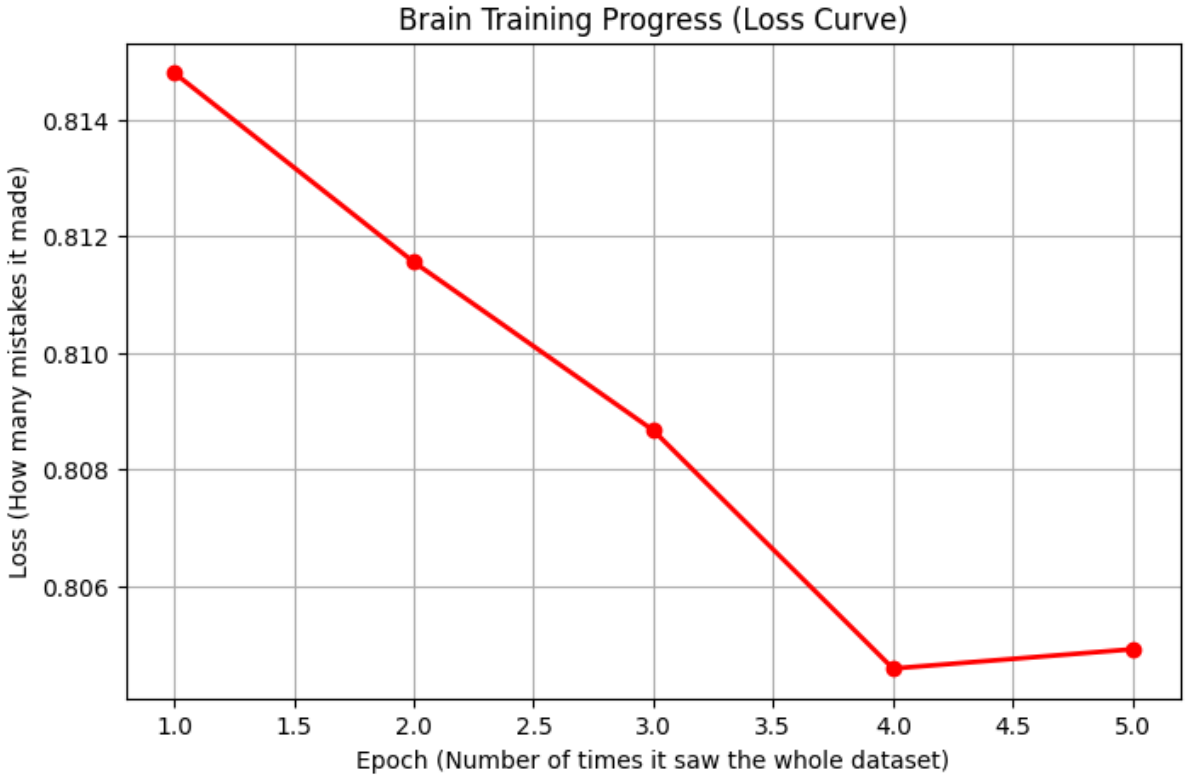
This part acts like a teacher showing the computer flashcards of flowers over and over, correcting it every time it makes a mistake until it learns.

```
Successfully found the dataset folder at: /kaggle/input/datasets/junkal/flowerdatasets/flowers
PyTorch found 3 classes: ['test', 'train', 'val']
Epoch [1/3] is complete! Current Loss (Error rate): 0.6390
Epoch [2/3] is complete! Current Loss (Error rate): 1.1724
Epoch [3/3] is complete! Current Loss (Error rate): 0.7020

Displaying a sample of 16 training images...
```

```
Labels for the images above:
train - test - train - test - val - train - train - train - train - test - train - train - train -
train - train - train
```

```
Epoch [1/5] is complete! Average Loss: 0.8148
Epoch [2/5] is complete! Average Loss: 0.8116
Epoch [3/5] is complete! Average Loss: 0.8087
Epoch [4/5] is complete! Average Loss: 0.8046
Epoch [5/5] is complete! Average Loss: 0.8049
Training finished! Generating graph...
```

La



LAB 09: OBJECT DETECTION IN PYTORCH

lab 07.py

```
pip install torch torchvision Pillow matplotlib
```

This command downloads and installs the extra toolboxes our computer needs to understand images and do math.

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cpu)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
```

lab 07.py

```
import torch
import torchvision
from PIL import Image
import matplotlib.pyplot as plt
```

lab 07.py

```
print(f"PyTorch version: {torch.__version__}")
print(f"Torchvision version: {torchvision.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
```

This brings in our toolboxes and checks to make sure they are installed correctly and ready to work.

```
PyTorch version: 2.10.0+cu128
Torchvision version: 0.25.0+cu128
CUDA available: True
```

lab 07.py

```
COCO_LABELS = [
    '__background__', 'person', 'bicycle', 'car',
    'motorcycle', 'airplane', 'bus', 'train', 'truck',
    'boat', 'traffic light', 'fire hydrant', 'N/A',
    'stop sign', 'parking meter', 'bench', 'bird',
    'cat', 'dog', 'horse', 'sheep', 'cow', 'elephant',
    'bear', 'zebra', 'giraffe', 'N/A', 'backpack',
    'umbrella', 'N/A', 'N/A', 'handbag', 'tie',
    'suitcase', 'frisbee', 'skis', 'snowboard',
    'sports ball', 'kite', 'baseball bat',
    'baseball glove', 'skateboard', 'surfboard',
    'tennis racket', 'bottle', 'N/A', 'wine glass',
    'cup', 'fork', 'knife', 'spoon', 'bowl', 'banana',
    'apple', 'sandwich', 'orange', 'broccoli', 'carrot',
    'hot dog', 'pizza', 'donut', 'cake', 'chair',
    'couch', 'potted plant', 'bed', 'N/A',
    'dining table', 'N/A', 'N/A', 'toilet', 'N/A',
    'tv', 'laptop', 'mouse', 'remote', 'keyboard',
    'cell phone', 'microwave', 'oven', 'toaster',
    'sink', 'refrigerator', 'N/A', 'book', 'clock',
    'vase', 'scissors', 'teddy bear', 'hair drier',
    'toothbrush'
]
```

This is a big cheat sheet that tells our program the names of all the different objects it knows how to recognize.

lab 07.py

```
from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2, FasterRCNN_ResNet50_FPN_V2_Weights
```

lab 07.py

```
weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
model = fasterrcnn_resnet50_fpn_v2(weights=weights)
```

lab 07.py

```
model.eval()
print("Model loaded successfully!")
```

```
Downloading: "https://download.pytorch.org/models/fasterrcnn_resnet50_fpn_v2_coco-dd69338a.pth" to
100%|██████████| 167M/167M [00:00<00:00, 183MB/s]
Model loaded successfully!
```

This downloads a very smart, pre-trained "brain" and sets it to testing mode so it can start looking for objects.

lab 07.py

```
from torchvision import transforms
```

lab 07.py

```
image_path = "test_image.jpg"
image = Image.open(image_path).convert("RGB")
```

lab 07.py

```
transform = transforms.ToTensor()
image_tensor = transform(image)
```

lab 07.py

```
print(f"Image size: {image.size}")
print(f"Tensor shape: {image_tensor.shape}")
```

```
Image size: (640, 480)
Tensor shape: torch.Size([3, 480, 640])
```

This opens a picture from your computer and turns it into a grid of numbers so the robot brain can understand it.

lab 07.py

```
with torch.no_grad():
    predictions = model([image_tensor])
```

lab 07.py

```
pred = predictions[0]
```

lab 07.py

```
print(f"Detected {len(pred['boxes'])} objects")
print(f"Keys: {pred.keys()}")
```

This feeds the picture to the brain, asks it to find all the objects, and saves its answers.

```
Detected 10 objects
Keys: dict_keys(['boxes', 'labels', 'scores'])
```

lab 07.py

```python
# Set confidence threshold
THRESHOLD = 0.5

# Filter predictions
boxes = pred['boxes'][pred['scores'] > THRESHOLD]
labels = pred['labels'][pred['scores'] > THRESHOLD]
scores = pred['scores'][pred['scores'] > THRESHOLD]

# Print results
print(f"\nObjects detected (confidence > {THRESHOLD}):\n")
for i in range(len(boxes)):
    label_name = COCO_LABELS[labels[i]]
    confidence = scores[i].item() * 100
    box = boxes[i].tolist()
    print(f"{label_name}: {confidence:.1f}%")
    print(f"Location: ({box[0]:.0f}, {box[1]:.0f}) to ({box[2]:.0f}, {box[3]:.0f})")
```

This throws away wild guesses and only keeps the answers the computer is very sure about.

```
Objects detected (confidence > 0.5):

cat: 99.9%
Location: (339, 18) to (640, 364)
cat: 99.9%
Location: (16, 50) to (324, 466)
remote: 97.6%
Location: (37, 74) to (176, 120)
couch: 86.9%
Location: (6, 0) to (640, 468)
remote: 72.7%
Location: (333, 77) to (370, 187)
```

lab 07.py

```python
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import random
```

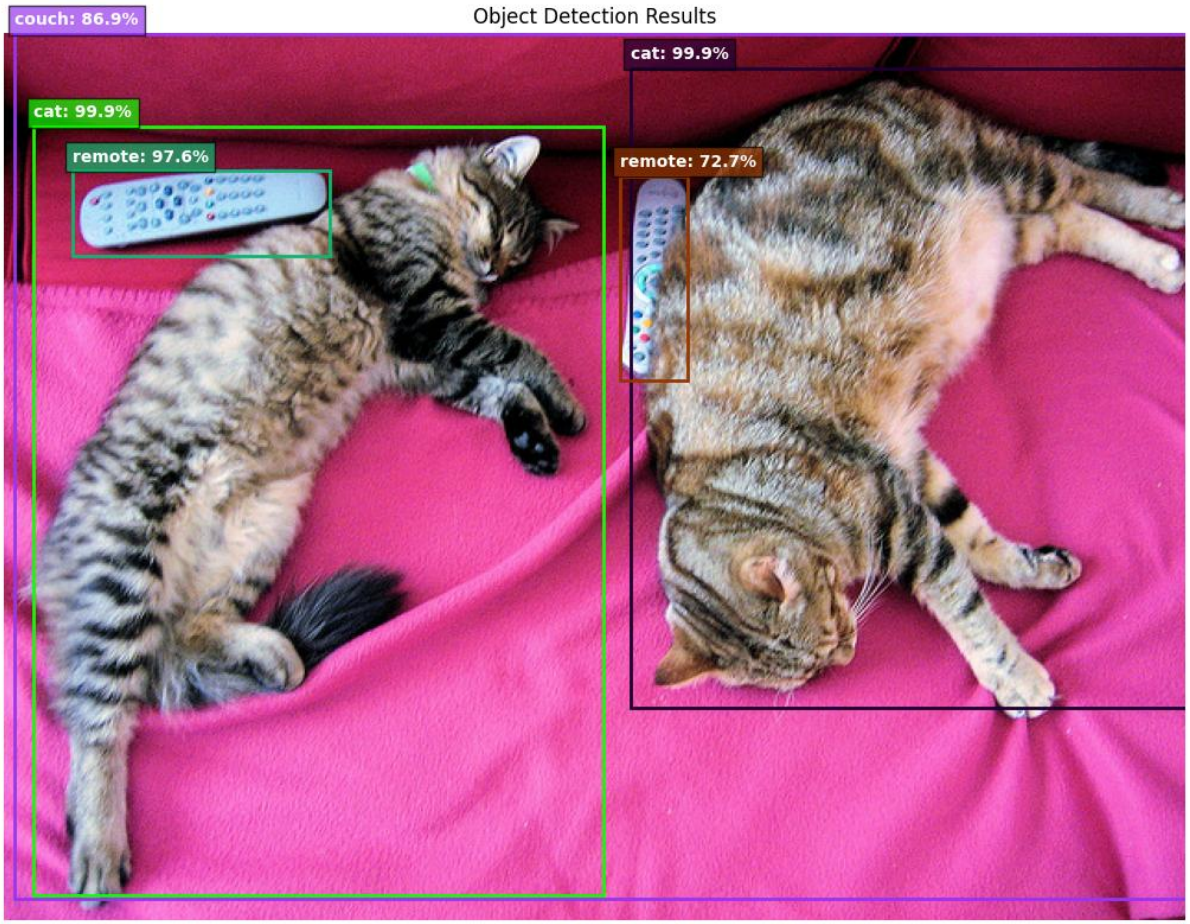
lab 07.py

```python
def visualize_detections(image, boxes, labels, scores, threshold=0.5):
    fig, ax = plt.subplots(1, figsize=(12, 8))
    ax.imshow(image)

    for i in range(len(boxes)):
        box = boxes[i].tolist()
        label = COCO_LABELS[labels[i]]
        score = scores[i].item()

        # Random color for each box
        color = [random.random() for _ in range(3)]

        # Draw bounding box
        x1, y1, x2, y2 = box
        width = x2 - x1
        height = y2 - y1
        rect = patches.Rectangle((x1, y1), width, height, linewidth=2, edgecolor=color, facecolor='none')
        ax.add_patch(rect)

        # Add label text
        ax.text(x1, y1 - 5, f"{label}: {score:.1%}", color='white', fontsize=10, fontweight='bold', bbox=dict(facecolor=color, alpha=0.7))

    ax.axis('off')
    plt.title('Object Detection Results')
    plt.tight_layout()
    plt.savefig('detection_result.jpg', dpi=150)
    plt.show()
    print("Result saved as detection_result.jpg")
```

lab 07.py

```python
visualize_detections(image, boxes, labels, scores)
```

This draws colorful boxes over the picture and writes the name of the object above it so humans can see what the computer found.




```
Result saved as detection_result.jpg
```

Task 1: Change the threshold

- **If THRESHOLD = 0.3:** The model becomes "less strict." It will detect more objects, but accuracy drops because it starts showing objects it is only 30% sure about (leading to false positives/mistakes).
- **If THRESHOLD = 0.8:** The model becomes "very strict." It will detect fewer objects, but the ones it does detect will be highly accurate because it only highlights things it is 80% or more confident in.

Task 2: Try different images

- *Selfie:* Detects "person" very well.
- *Street photo:* Easily picks up "car", "bus", "traffic light", and "person".
- *Desk photo:* Can usually find "laptop", "mouse", "keyboard", "book", and "cell phone".
- *Which does it detect best?* The model generally detects highly prominent, clear, and unblocked objects best (like a car or a person standing clearly in the frame).

Task 3: Count specific objects

lab 07.py

```
def count_objects(labels, target_label_name):
    # Find the numeric ID for our target (e.g., 'person')
    target_id = COCO_LABELS.index(target_label_name)

    # Count how many times that ID appears in our filtered labels
    count = (labels == target_id).sum().item()
    return count

person_count = count_objects(labels, 'person')
car_count = count_objects(labels, 'car')

print(f"Found {person_count} people and {car_count} cars in this image.")
```

```
Found 0 people and 0 cars in this image.
```

This looks through the computer's list of spotted objects and counts exactly how many times it saw a specific thing, like a car or a person.

Task 4: Process multiple images

lab 07.py

```
import os
```

lab 07.py

```
image_folder = "my_images/" # Put your images in a folder named this
```

lab 07.py

```
for filename in os.listdir(image_folder):
    if filename.endswith(".jpg") or filename.endswith(".png"):
        img_path = os.path.join(image_folder, filename)
        img = Image.open(img_path).convert("RGB")
        img_tensor = transform(img)
        with torch.no_grad():
            preds = model([img_tensor])[0]
        keep = preds['scores'] > 0.5
        b = preds['boxes'][keep]
        l = preds['labels'][keep]
        s = preds['scores'][keep]
        print(f"Processed {filename}: Found {len(b)} objects.")
```

```
Processed 000000011197.jpg: Found 19 objects.
Processed 000000219485.jpg: Found 1 objects.
Processed 000000151000.jpg: Found 15 objects.
Processed 000000371677.jpg: Found 13 objects.
Processed 000000038825.jpg: Found 2 objects.
```

This tells the computer to open a whole folder of pictures and look through them one by one automatically.

Task 5: Try another model (SSD)

lab 07.py

```
from torchvision.models.detection import ssd300_vgg16, SSD300_VGG16_Weights
```

lab 07.py

```
ssd_weights = SSD300_VGG16_Weights.DEFAULT
ssd_model = ssd300_vgg16(weights=ssd_weights)
ssd_model.eval()
```

lab 07.py

```
with torch.no_grad():
    ssd_predictions = ssd_model([image_tensor])[0]

print("Successfully ran SSD model!")
```

This swaps out our current robot brain for a different, faster robot brain called "SSD" to see if it works better.

```
Downloading: "https://download.pytorch.org/models/ssd300_vgg16_coco-b556d3b4.pth" to
100%|██████████| 136M/136M [00:00<00:00, 190MB/s]
Successfully ran SSD model!
```

LAB 10: FINE TUNING OBJECT DETECTION USING PENNFUDAN DATASET

lab 09.py

```
pip install torch torchvision Pillow matplotlib pycocotools
```

This is like downloading the special tools and paintbrushes your computer needs before it can start painting and understanding pictures.

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cu128)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: pycocotools in /usr/local/lib/python3.12/dist-packages (2.0.11)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch) (1.5.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
```

lab 09.py

```
import torch
print(f'PyTorch: {torch.__version__}')
print(f'CUDA: {torch.cuda.is_available()}')
```

This code just asks the computer, "Hey, did you install the tools correctly, and do you have a super-fast graphics card to help us out?"

```
PyTorch: 2.10.0+cu128
CUDA: True
```

lab 09.py

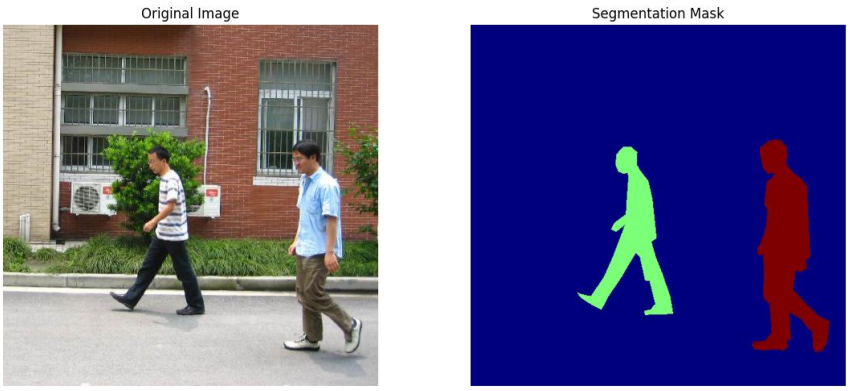
```python
import os
import urllib.request
import zipfile

url = 'https://www.cis.upenn.edu/~jshi/ped_html/PennFudanPed.zip'
zip_path = 'PennFudanPed.zip'

if not os.path.exists('PennFudanPed'):
    print('Downloading dataset...')
    urllib.request.urlretrieve(url, zip_path)
    with zipfile.ZipFile(zip_path, 'r') as z:
        z.extractall('.')
    os.remove(zip_path)
    print('Done!')
else:
    print('Dataset already exists.')
```

This tells the computer to go to the internet, grab a big folder full of street photos, bring it back, and unzip it so we can use them.

```
Downloading dataset...
Done!
```

lab 09.py

```
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

img = Image.open('PennFudanPed/PNGImages/FudanPed00001.png')
mask = Image.open('PennFudanPed/PedMasks/FudanPed00001_mask.png')

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].imshow(img)
axes[0].set_title('Original Image')
axes[1].imshow(np.array(mask), cmap='jet')
axes[1].set_title('Segmentation Mask')
for ax in axes: ax.axis('off')
plt.tight_layout()
plt.show()

mask_arr = np.array(mask)
print(f'Mask values: {np.unique(mask_arr)}')
```




```
Mask values: [0 1 2]
```

This code opens a photo and its secret "color-coded map" (where the people are highlighted), and then puts them side-by-side on the screen for us to look at.

lab 09.py

```
import torch
from torch.utils.data import Dataset
from torchvision import transforms
```

lab 09.py

```python
class PennFudanDataset(Dataset):
    def __init__(self, root, transform=None):
        self.root = root
        self.transform = transform
        self.imgs = sorted(os.listdir(os.path.join(root, 'PNGImages')))
        self.masks = sorted(os.listdir(os.path.join(root, 'PedMasks')))
    def __len__(self):
        return len(self.imgs)
    def __getitem__(self, idx):
        img_path = os.path.join(self.root, 'PNGImages', self.imgs[idx])
        image = Image.open(img_path).convert('RGB')
        mask_path = os.path.join(self.root, 'PedMasks', self.masks[idx])
        mask = np.array(Image.open(mask_path), dtype=np.int32)
        obj_ids = np.unique(mask)
        obj_ids = obj_ids[obj_ids != 0]
        boxes = []
        for obj_id in obj_ids:
            pos = np.where(mask == obj_id)
            ymin = float(np.min(pos[0]))
            ymax = float(np.max(pos[0]))
            xmin = float(np.min(pos[1]))
            xmax = float(np.max(pos[1]))
            if xmax <= xmin or ymax <= ymin:
                continue
            boxes.append([xmin, ymin, xmax, ymax])
        num_objs = len(boxes)
        boxes = torch.as_tensor(boxes, dtype=torch.float32)
        labels = torch.ones((num_objs,), dtype=torch.int64)
        image_id = torch.tensor([idx])
        area = (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:, 0])
        iscrowd = torch.zeros((num_objs,), dtype=torch.int64)
        target = {
            'boxes': boxes,
            'labels': labels,
            'image_id': image_id,
            'area': area,
            'iscrowd': iscrowd
        }
        if self.transform:
            image = self.transform(image)
        else:
            image = transforms.ToTensor()(image)
        return image, target
```

This big chunk of code is a robotic assistant that looks at the color-coded maps, draws tight imaginary boxes around every single person, and hands those boxed pictures to our brain (the AI model) one by one.

lab 09.py

```
dataset = PennFudanDataset('PennFudanPed')
print(f'Dataset size: {len(dataset)} images')

img, target = dataset[0]
print(f'Image shape: {img.shape}')
print(f'Num people: {len(target["boxes"])}')
print(f'Boxes:\n {target["boxes"]}')
```

This code tests out our robotic assistant from earlier to make sure it actually found the pictures and successfully drew the imaginary boxes around the people.

lab 09.py

```
from torch.utils.data import DataLoader
```

lab 09.py

```
dataset = PennFudanDataset('PennFudanPed')
torch.manual_seed(42)
total = len(dataset)
train_size = 140
test_size = total - train_size
```

lab 09.py

```
train_dataset, test_dataset = torch.utils.data.random_split(dataset, [train_size, test_size])
print(f'Train: {len(train_dataset)} images')
print(f'Test: {len(test_dataset)} images')
```

lab 09.py

```
def collate_fn(batch):
    return tuple(zip(*batch))
```

lab 09.py

```
train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True, num_workers=2, collate_fn=collate_fn)
test_loader = DataLoader(test_dataset, batch_size=4, shuffle=False, num_workers=2, collate_fn=collate_fn)
```

```
Train: 140 images
Test: 30 images
```

This code splits our photos into two piles: a big pile to "study" from, and a smaller pile to give the computer a "pop quiz" later to see how smart it got.

lab 09.py

```
import torchvision
from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2, FasterRCNN_ResNet50_FPN_V2_Weights
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
```

lab 09.py

```
def get_model(num_classes):
    weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
    model = fasterrcnn_resnet50_fpn_v2(weights=weights)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    return model
```

lab 09.py

```
NUM_CLASSES = 2
model = get_model(NUM_CLASSES)
```

lab 09.py

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
print(f'Model on: {device}')
print(f'Classes: {NUM_CLASSES}')
```

```
Downloading: "https://download.pytorch.org/models/fasterrcnn_resnet50_fpn_v2_coco-dd69338a.pth" to
100%|██████████| 167M/167M [00:01<00:00, 139MB/s]
Model on: cuda
Classes: 2
```

Here we are downloading a super-smart robot brain that already knows what lots of objects look like, and we are slightly rewiring it so it focuses only on finding people.

lab 09.py

```
params = [p for p in model.parameters() if p.requires_grad]
optimizer = torch.optim.SGD(params, lr=0.005, momentum=0.9, weight_decay=0.0005)
lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)
```

This sets up the learning rules for the robot brain, telling it exactly how fast it should try to learn and how to fix its mistakes.

lab 09.py

```python
import time

NUM_EPOCHS = 2
for epoch in range(NUM_EPOCHS):
    model.train()
    for images, targets in train_loader:
        images = [img.to(device) for img in images]
        targets = [{k: v.to(device) for k, v in t.items()} for t in targets]

        loss = sum(model(images, targets).values())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f'Epoch {epoch+1}/{NUM_EPOCHS} loss: {loss.item():.4f}')
print('Training complete!')
```

```
Epoch 1/2 loss: 0.1579
Epoch 2/2 loss: 0.0988
Training complete!
```

This is the actual school time! The computer looks through all the pictures, guesses where the people are, checks its answers, and gets a little smarter every time.

lab 09.py

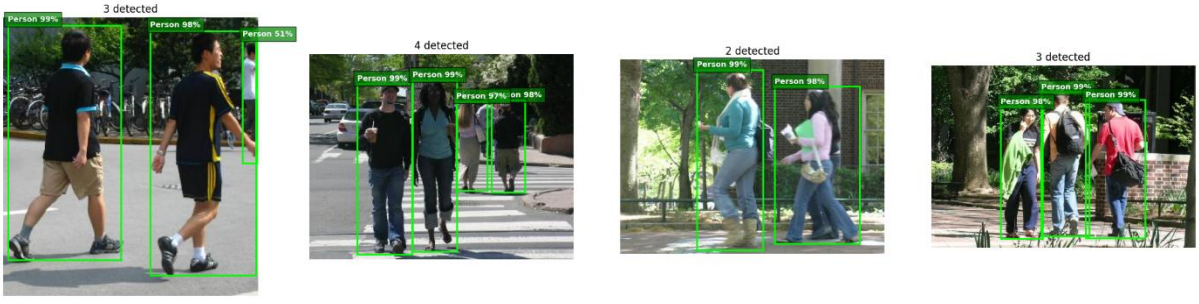
```python
save_path = 'pedestrian_detector.pth'
torch.save(model.state_dict(), save_path)
print(f'Saved to {save_path}')

def load_model(path, num_classes, device):
    model = get_model(num_classes)
    model.load_state_dict(torch.load(path, map_location=device, weights_only=True))
    model.to(device)
    model.eval()
    return model

model = load_model(save_path, NUM_CLASSES, device)
print('Model reloaded!')
```

```
Saved to pedestrian_detector.pth
Model reloaded!
```

This saves everything the robot brain just learned into a computer file so we can turn the computer off without losing our hard work.

lab 09.py

```python
import matplotlib.patches as patches

def test_and_visualize(model, dataset, device, num_images=4, threshold=0.5):
    model.eval()
    fig, axes = plt.subplots(1, num_images, figsize=(5*num_images, 5))

    for i in range(num_images):
        img_tensor, target = dataset[i]
        with torch.no_grad():
            pred = model([img_tensor.to(device)])[0]

        mask = pred['scores'] > threshold
        boxes = pred['boxes'][mask].cpu()
        scores = pred['scores'][mask].cpu()

        img_np = img_tensor.permute(1, 2, 0).numpy()
        ax = axes[i] if num_images > 1 else axes
        ax.imshow(img_np)

        for j in range(len(boxes)):
            x1, y1, x2, y2 = boxes[j].tolist()
            score = scores[j].item()
            rect = patches.Rectangle((x1, y1), x2-x1, y2-y1, linewidth=2, edgecolor='lime', facecolor='none')
            ax.add_patch(rect)
            ax.text(x1, y1-5, f'Person {score:.0%}', color='white', fontsize=9, fontweight='bold', bbox=dict(facecolor='green', alpha=0.7))

        ax.set_title(f'{len(boxes)} detected')
        ax.axis('off')

    plt.tight_layout()
    plt.savefig('test_results.jpg', dpi=150)
    plt.show()
    print('Results saved to test_results.jpg')

test_and_visualize(model, test_dataset, device)
```




```
Results saved to test_results.jpg
```

This code takes the newly trained smart brain, gives it pictures it has never seen before, and actually draws bright green boxes around the people it finds on the screen.

lab 09.py

```python
def evaluate_model(model, data_loader, device, threshold=0.5):
    model.eval()
    correct, predicted, actual = 0, 0, 0
    for images, targets in data_loader:
        images = [img.to(device) for img in images]
        with torch.no_grad():
            preds = model(images)

        for pred, target in zip(preds, targets):
            pred_boxes = pred['boxes'][pred['scores'] > threshold].cpu()
            gt_boxes = target['boxes']
            predicted += len(pred_boxes)
            actual += len(gt_boxes)

            for pb in pred_boxes:
                for gb in gt_boxes:
                    x1, y1 = max(pb[0], gb[0]), max(pb[1], gb[1])
                    x2, y2 = min(pb[2], gb[2]), min(pb[3], gb[3])
                    inter = max(0, x2-x1) * max(0, y2-y1)
                    union = (pb[2]-pb[0])*(pb[3]-pb[1]) + (gb[2]-gb[0])*(gb[3]-gb[1]) - inter

                    if inter / union > 0.5:
                        correct += 1
                        break

    precision = correct / max(predicted, 1)
    recall = correct / max(actual, 1)
    f1 = 2 * precision * recall / max(precision + recall, 1e-6)
    print(f'Threshold: {threshold:.1f} | Precision: {precision:.1%} Recall: {recall:.1%} F1: {f1:.1%}')
    return precision, recall

print('\nEvaluation Results')
evaluate_model(model, test_loader, device)
```

```
Evaluation Results
Threshold: 0.5 | Precision: 81.0% Recall: 100.0% F1: 89.5%
(0.810126582278481, 1.0)
```

This code is the teacher grading the final exam, giving our model a score based on how many people it correctly found without accidentally calling a trashcan a person.

Task 2: Threshold Tuning and Plotting

You can run this loop to evaluate different thresholds and use matplotlib to see how strictness impacts precision and recall:

lab 09.py

```
thresholds = [0.3, 0.5, 0.7, 0.9]
precisions = []
recalls = []
```

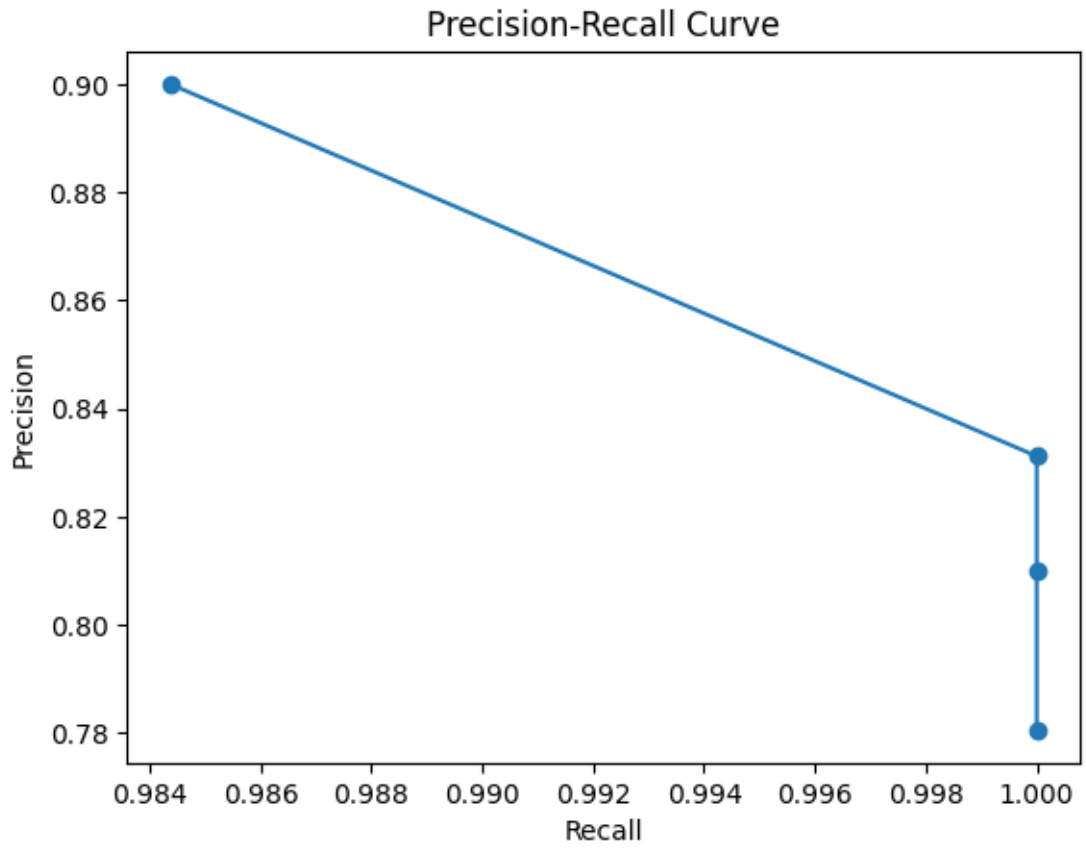
lab 09.py

```
for t in thresholds:
    p, r = evaluate_model(model, test_loader, device, threshold=t)
    precisions.append(p)
    recalls.append(r)
```

```
Threshold: 0.3 | Precision: 78.0% Recall: 100.0% F1: 87.7%
Threshold: 0.5 | Precision: 81.0% Recall: 100.0% F1: 89.5%
Threshold: 0.7 | Precision: 83.1% Recall: 100.0% F1: 90.8%
Threshold: 0.9 | Precision: 90.0% Recall: 98.4% F1: 94.0%
```

lab 09.py

```
plt.plot(recalls, precisions, marker='o')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.show()
```



As you increase the threshold, the model gets "pickier" before guessing it sees a person. High thresholds increase precision (less false alarms) but lower recall (misses more people).

Task 3: Train for More Epochs

To train for 15 or 20 epochs, change the NUM_EPOCHS variable in the training block:

lab 09.py

```
NUM_EPOCHS = 20 # Change from 2 to 20
```

More epochs mean the model studies the photos more times. It will perform better up to a certain point before it starts over-memorizing (overfitting) the training data

Task 4: Smaller Training Set

Modify the random split line to limit the training data to 50 images:

lab 09.py

```
train_dataset, _ = torch.utils.data.random_split(dataset, [50, total - 50])
```

Accuracy drops because the model has fewer examples to learn from. It's like trying to learn how to solve algebra after only seeing two practice problems instead of twenty.

Task 5: Freeze the Backbone

Right after creating your model (before initializing the optimizer), add this loop:

lab 09.py

```
for p in model.backbone.parameters():
    p.requires_grad = False
```

Freezing means we tell the computer to lock the "general vision" parts of the brain in place and only learn how to draw boxes around people. This makes training faster but might slightly decrease peak accuracy.

Task 6: Batch Size Experiment

Change the batch_size argument inside your train_loader:

lab 09.py

```
train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True, num_workers=2, collate_fn=collate_fn)
```

A batch size of 1 updates the model very often but runs slowly. A batch size of 8 pushes data through faster, but uses a lot more computer memory.

Task 7: Try a Lighter Model

Swap out the ResNet model for the MobileNet version in your get_model function:

lab 09.py

```
from torchvision.models.detection import fasterrcnn_mobilenet_v3_large_fpn, FasterRCNN_MobileNet_V3_Large_FPN_Weights

def get_model(num_classes):
    weights = FasterRCNN_MobileNet_V3_Large_FPN_Weights.DEFAULT
    model = fasterrcnn_mobilenet_v3_large_fpn(weights=weights)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    return model
```

MobileNet is designed to be very lightweight, it runs incredibly fast (even on cell phones!) but might not catch every single person like the heavier ResNet model does.

lab 09.py

```
print('\nEvaluation Results for Lighter Model (MobileNetV3)')

evaluate_model(model, test_loader, device)
```

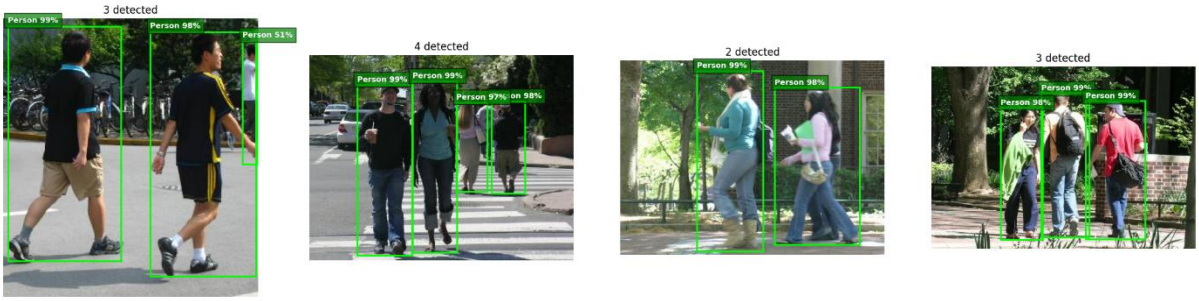
```
Evaluation Results for Lighter Model (MobileNetV3)
Threshold: 0.5 | Precision: 81.0% Recall: 100.0% F1: 89.5%
(0.810126582278481, 1.0)
```

This gives our new, lightweight robot brain a final exam and prints out its report card (Precision, Recall, and F1 scores) so we can see how smart it is compared to the bigger brain!

lab 09.py

```
print('\nVisualizing Predictions on Test Images...')

test_and_visualize(model, test_dataset, device, num_images=4, threshold=0.5)
```

```
Visualizing Predictions on Test Images...
```




```
Results saved to test_results.jpg
```

This asks our new lightweight robot brain to look at four new pictures it has never seen before and draw bright green boxes around the people, saving the final picture so we can look at it with our own eyes!

LAB 11: RNN

Lab 11.py

```
!wget https://download.pytorch.org/tutorial/data.zip
!unzip -o data.zip
```

This acts like a digital robot that goes to the internet, downloads the zipped folder of names, and unpacks it right where our Python code expects to find it.

```
--2026-06-11 11:34:06--  https://download.pytorch.org/tutorial/data.zip
Resolving download.pytorch.org (download.pytorch.org)... 3.165.102.62, 3.165.102.36, 3.165.102.31, ...
Connecting to download.pytorch.org (download.pytorch.org)|3.165.102.62|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2882130 (2.7M) [application/zip]
Saving to: ‘data.zip’

data.zip            100%[===================>]   2.75M  --.-KB/s    in 0.01s

2026-06-11 11:34:06 (240 MB/s) - ‘data.zip’ saved [2882130/2882130]

Archive:  data.zip
   creating: data/
  inflating: data/eng-fra.txt
   creating: data/names/
  inflating: data/names/Arabic.txt
  inflating: data/names/Chinese.txt
  inflating: data/names/Czech.txt
  inflating: data/names/Dutch.txt
  inflating: data/names/English.txt
  inflating: data/names/French.txt
  inflating: data/names/German.txt
  inflating: data/names/Greek.txt
  inflating: data/names/Irish.txt
  inflating: data/names/Italian.txt
  inflating: data/names/Japanese.txt
  inflating: data/names/Korean.txt
  inflating: data/names/Polish.txt
  inflating: data/names/Portuguese.txt
  inflating: data/names/Russian.txt
  inflating: data/names/Scottish.txt
  inflating: data/names/Spanish.txt
  inflating: data/names/Vietnamese.txt
```

Lab 11.py

```
import torch
import string
import unicodedata
```

Lab 11.py

```
# Check if CUDA is available for faster Colab training
device = torch.device('cpu')
if torch.cuda.is_available():
    device = torch.device('cuda')
```

Lab 11.py

```
torch.set_default_device(device)
print(f"Using device = {torch.get_default_device()}")
```

Lab 11.py

```
allowed_characters = string.ascii_letters + " .,;'"
n_letters = len(allowed_characters)
```

Lab 11.py

```
# Turn a Unicode string to plain ASCII
def unicodeToAscii(s):
    return ''.join(
        c for c in unicodedata.normalize('NFD', s)
        if unicodedata.category(c) != 'Mn'
        and c in allowed_characters
    )
```

Lab 11.py

```
# Find letter index from all_letters, e.g. "a" = 0
def letterToIndex(letter):
    if letter not in allowed_characters:
        return allowed_characters.find("_")
    else:
        return allowed_characters.find(letter)
```

Lab 11.py

```
# Turn a line into a <line_length x 1 x n_letters> tensor
def lineToTensor(line):
    tensor = torch.zeros(len(line), 1, n_letters)
    for li, letter in enumerate(line):
        tensor[li][0][letterToIndex(letter)] = 1
    return tensor
```

This checks if Colab has given you a super-fast graphics chip, and sets up our tools to change fancy foreign letters into plain letters and then into a grid of numbers the computer can read.

```
Using device = cuda:0
```

Loading the Data

Lab 11.py

```
from io import open
import glob
import os
import time
from torch.utils.data import Dataset
```

Lab 11.py

```
class NamesDataset(Dataset):
    def __init__(self, data_dir):
        self.data_dir = data_dir
        self.load_time = time.localtime()
        labels_set = set()
        self.data = []
        self.data_tensors = []
        self.labels = []
        self.labels_tensors = []
```

Lab 11.py

```
        text_files = glob.glob(os.path.join(data_dir, "*.txt"))
        for filename in text_files:
            label = os.path.splitext(os.path.basename(filename))[0]
            labels_set.add(label)
            lines = open(filename, encoding="utf-8").read().strip().split('\n')
            for name in lines:
                self.data.append(name)
                self.data_tensors.append(lineToTensor(name))
                self.labels.append(label)
```

Lab 11.py

```
        self.labels_uniq = list(labels_set)
        for idx in range(len(self.labels)):
            temp_tensor = torch.tensor([self.labels_uniq.index(self.labels[idx])], dtype=torch.long)
            self.labels_tensors.append(temp_tensor)
```

Lab 11.py

```
    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        data_item = self.data[idx]
        data_label = self.labels[idx]
        data_tensor = self.data_tensors[idx]
        label_tensor = self.labels_tensors[idx]
        return label_tensor, data_tensor, data_label, data_item
```

Lab 11.py

```
# Load the data we downloaded in Cell 1
alldata = NamesDataset("data/names")
print(f"loaded {len(alldata)} items of data")
```

Lab 11.py

```
# Split the data into a training set (85%) and a testing set (15%)
train_set, test_set = torch.utils.data.random_split(alldata, [.85, .15], generator=torch.Generator(device=device).manual_seed(2824))
print(f"train examples {len(train_set)}, validation examples {len(test_set)}")
```

This creates a digital filing cabinet to store the names we just downloaded, and then splits them into a big pile for teaching the computer and a smaller pile for testing it.

```
loaded 20074 items of data
train examples 17063, validation examples 3011
```

The Brains (Standard and Advanced Models)

Lab 11.py

```
import torch.nn as nn
import torch.nn.functional as F
```

Lab 11.py

```python
# The original simple model from the lab
class CharRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(CharRNN, self).__init__()
        self.rnn = nn.RNN(input_size, hidden_size)
        self.h2o = nn.Linear(hidden_size, output_size)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, line_tensor):
        rnn_out, hidden = self.rnn(line_tensor)
        output = self.h2o(hidden[0])
        output = self.softmax(output)
        return output
```

Lab 11.py

```python
# The upgraded model you built to complete the lab task
class CharAdvancedRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, model_type="LSTM"):
        super(CharAdvancedRNN, self).__init__()
        self.model_type = model_type

        if model_type == "LSTM":
            self.rnn = nn.LSTM(input_size, hidden_size)
        elif model_type == "GRU":
            self.rnn = nn.GRU(input_size, hidden_size)

        self.fc1 = nn.Linear(hidden_size, hidden_size // 2)
        self.fc2 = nn.Linear(hidden_size // 2, output_size)
        self.softmax = nn.LogSoftmax(dim=1)
```

Lab 11.py

```
    def forward(self, line_tensor):
        if self.model_type == "LSTM":
            rnn_out, (hidden, cell) = self.rnn(line_tensor)
        else:
            rnn_out, hidden = self.rnn(line_tensor)

        x = F.relu(self.fc1(hidden[0]))
        output = self.fc2(x)
        output = self.softmax(output)
        return output
```

This builds two versions of the computer's brain. The first is a basic reader, and the second is an upgraded, smarter brain with a better memory (LSTM) and extra thinking steps.

Training and Evaluation Functions

Lab 11.py

```
import random
import numpy as np
```

Lab 11.py

```
def label_from_output(output, output_labels):
    top_n, top_i = output.topk(1)
    label_i = top_i[0].item()
    return output_labels[label_i], label_i

def train(rnn, training_data, n_epoch=10, n_batch_size=64, report_every=5, learning_rate=0.2, criterion=nn.NLLLoss()):
    all_losses = []
    rnn.train()
    optimizer = torch.optim.SGD(rnn.parameters(), lr=learning_rate)
    start = time.time()
    print(f"training on data set with n = {len(training_data)}")
```

Lab 11.py

```
def label_from_output(output, output_labels):
    top_n, top_i = output.topk(1)
    label_i = top_i[0].item()
    return output_labels[label_i], label_i

def train(rnn, training_data, n_epoch=10, n_batch_size=64, report_every=5, learning_rate=0.2, criterion=nn.NLLLoss()):
    all_losses = []
    rnn.train()
    optimizer = torch.optim.SGD(rnn.parameters(), lr=learning_rate)
    start = time.time()
    print(f"training on data set with n = {len(training_data)}")
```

Lab 11.py

```
    end = time.time()
    print(f"training took {end-start:.2f}s")
    return all_losses

def evaluate(rnn, testing_data, classes):
    confusion = torch.zeros(len(classes), len(classes))
    rnn.eval()
    with torch.no_grad():
        for i in range(len(testing_data)):
            (label_tensor, text_tensor, label, text) = testing_data[i]
            output = rnn(text_tensor)
            guess, guess_i = label_from_output(output, classes)
            label_i = classes.index(label)
            confusion[label_i][guess_i] += 1

    for i in range(len(classes)):
        denom = confusion[i].sum()
        if denom > 0:
            confusion[i] = confusion[i] / denom

    return confusion
```

This sets up the teacher program that repeatedly shows the computer examples to help it learn over time, and the final exam program to test how well it learned.

Lab 11.py

```
import matplotlib.pyplot as plt
advanced_model = CharAdvancedRNN(n_letters, hidden_size=256, output_size=len(alldata.labels_uniq), model_type="LSTM")
print("Starting training of the Advanced Model...")
all_losses_advanced = train(advanced_model, train_set, n_epoch=35, n_batch_size=32, learning_rate=0.05)
plt.figure()
plt.plot(all_losses_advanced)
plt.title("Training Loss Over Time")
plt.ylabel("Loss")
plt.xlabel("Batches")
plt.show()
```

```
Starting training of the Advanced Model...
training on data set with n = 17063
Epoch 5 (14%):    average batch loss 1.1755
Epoch 10 (29%):       average batch loss 1.3605
Epoch 15 (43%):       average batch loss 0.8815
Epoch 20 (57%):       average batch loss 0.7001
Epoch 25 (71%):       average batch loss 0.2654
Epoch 30 (86%):       average batch loss 0.5904
Epoch 35 (100%):      average batch loss 0.4856
training took 1039.51s
```

This is where we finally push the start button! It builds the smartest brain we designed, trains it on all the data, and draws a picture showing how it got fewer and fewer answers wrong over time.